

Weekend Planner

Complete MVP Documentation

Version 2.0 | October 5, 2026

Owner: Agesta (Solo Founder)

PRD — Weekend Planner: Decision Engine App

Version: 2.0 (Updated after validation)

Date: October 5, 2026

Status: Final for MVP Development

Owner: Agesta (Founder & Solo Developer)

Executive Summary

Weekend Planner is a mobile-first web app that solves **decision fatigue** for weekend planning. Instead of overwhelming users with information, we take over the decision by generating 5 personalized venue/activity recommendations based on a 30-second quiz.

Core Value Prop: "Answer 5 questions → Get 5 perfect weekend spots"

1. Problem Statement

The Real Problem

Users aren't lacking information about weekend destinations — Google Maps, Instagram, and TikTok provide endless options. The real problem is **decision fatigue**: too many choices without a system to decide which option fits their current mood, budget, and context.

Evidence

- Survey feedback: 87% of respondents said "too many choices" is the main blocker
- Existing solutions (curated lists, social recommendations) require manual filtering
- No tool that "decides for you" based on context

2. Target Users

Primary

- **Demographics:** 20-35 years old, urban, middle income
- **Psychographics:** Value convenience over exhaustive research

- **Behavior:** Uses smartphone for planning, decides last-minute
- **Pain:** Spends 30+ minutes scrolling without deciding

Secondary (Post-MVP)

- Event planners looking for venue recommendations
- Corporate teams planning team building
- Tourism boards promoting local destinations

3. Goals & Success Metrics

North Star Metric

Weekly Active Recommendation Completions

= Users who complete quiz → view recommendations → take action (view maps/save)

Key Metrics

Acquisition:

- Landing page → Quiz start conversion: >40%
- Organic word-of-mouth coefficient: >0.2

Engagement:

- Quiz completion rate: >70%
- Recommendation click-through (Google Maps): >50%
- Regenerate rate: 20-35% (balance quality vs. dissatisfaction)

Retention:

- Day 2 return rate: >30%
- Day 7 return rate: >20%
- Weekly active users (WAU) growth: >15% week-over-week

Quality Indicators:

- Avg time on result page: >60 seconds
- Share rate: >5%
- Qualitative feedback sentiment: >70% positive

4. Scope - MVP Features

Must-Have (P0)

1. Quiz Flow

- **5-6 questions** (30-45 seconds to complete)
- Input dimensions:
 - Mood/vibe (santai, aktif, produktif, quality time)
 - Companion (sendiri, pasangan, teman, keluarga)
 - Budget per person (hemat <100k, menengah 100-300k, leluasa >300k)
 - Location scope (Jabodetabek, road trip, custom)
 - Time preference (pagi, siang, searian) - optional
 - Child age (conditional, if family selected)

2. AI Recommendation Engine

- Generate **5 recommendations** per quiz
- Powered by **Hermes-combo LLM** (via 9router API)
- Output format: JSON with name, category, reason, cost, location, best time

3. Result Display

- **5 recommendation cards** with:
 - Venue/activity name
 - Category badge (cafe, outdoor, kuliner, etc.)
 - Personalized reason why it fits
 - Budget estimate (range)
 - Location area
 - Best time to visit
- Actions per card:
 - "Lihat di Google Maps" (external link)
 - "Simpan" (requires login)

4. Authentication

- **Email + Password** registration
- Collect: email, password, phone number
- **Google OAuth** (Supabase Auth)
- Anonymous users can use 1 quota before forced login

5. Quota System

- **2 quota per user per day** (reset at 00:00 WIB)
- Anonymous: 1 quota (must login for quota #2)
- Registered: Full 2 quota access
- After exhausted: Show countdown to reset + history/share options
- **No regenerate with same quiz input** (each generate counts as quota)

6. Analytics Tracking

- Google Analytics 4 integration
- Track: page views, quiz completions, generations, auth events, quota exhaustion

Should-Have (P1 - Post-Soft Launch)

- View recommendation history (logged-in users)
- Save favorites (logged-in users)
- Share result page via link
- Improved mobile UX (swipeable cards, bottom sheet modals)

Won't Have (Out of Scope for MVP)

- ■ Booking/reservation integration
- ■ User reviews or ratings
- ■ Real-time venue availability
- ■ Photo uploads from users
- ■ Social features (follow, comments)
- ■ Premium/paid tier
- ■ Personalization based on history (simple history view only)

5. Technical Approach

Stack

- **Frontend:** Nuxt 3 (Vue 3, TypeScript, Tailwind CSS)
- **Backend:** Nuxt Server Routes (serverless functions)
- **Database:** Supabase (PostgreSQL + Auth)
- **LLM:** Hermes-combo via 9router API
- **Analytics:** Google Analytics 4
- **Hosting:** Vercel (frontend + serverless)

Architecture Pattern

- Monolithic Nuxt app (server + client in one repo)
- SSR for landing page (SEO)
- CSR for quiz and result pages (interactivity)
- API routes for LLM calls and database access

Security

- API keys stored in Vercel environment variables
- Row-level security (RLS) on Supabase
- Rate limiting: 2 quota/day per user (DB-enforced)
- Input sanitization on all user inputs

6. User Flows

Anonymous User (First Visit)

1. Land on homepage → Click "Mulai Quiz"
2. Complete 5-6 question quiz (30-45s)
3. Generate → Wait 3-5s → View 5 recommendations
4. Take action: View maps, or try to save (→ login prompt)
5. Want 2nd generation → Must login

Registered User

1. Login → Complete quiz
2. Generate (quota 1/2 used)

3. View results → Take action
4. Want more → Generate again (quota 2/2 used)
5. Quota exhausted → See countdown to reset + history/share options
6. Return next day → Quota reset to 2/2

Error/Edge Cases

- LLM generation fails → Show error, don't count quota, allow retry
- Network timeout → Show error, allow retry
- User clears localStorage → Lose anonymous session, can generate again
- User shares result link → Others can view (read-only)

7. Constraints & Assumptions

Constraints

- **Budget:** Limited (bootstrapped, no external funding yet)
- **Team:** Solo founder + AI assistants
- **Timeline:** MVP launch target: 2-3 weeks
- **LLM Cost:** Must stay <Rp 500k/month for first 100 users

Assumptions

- Hermes-combo LLM has sufficient knowledge of Indonesian venues
- 2 quota/day is enough for casual weekend planning
- Daily reset drives habit formation (return behavior)
- Users trust AI-generated recommendations without photos
- Phone number collection doesn't hurt conversion

Validation Plan

- Soft launch to 30-50 users (personal network)
- Monitor metrics for 2 weeks
- Interview 10 users for qualitative feedback
- Iterate based on data before scaling

8. Monetization Strategy (Deferred)

MVP Approach: Pure free model, daily reset, no paywalls

Future Monetization (Post-PMF):

- **Freemium:** Free tier (2/day) + Premium (unlimited, Rp 29-49k/month)
- **B2B:** Corporate event planning tier
- **Partnerships:** Commission from venue bookings
- **Sponsored recs:** Venues pay to be featured (clearly marked)

Decision Trigger (Add monetization when):

- 500+ registered users
- 30%+ DAU/MAU ratio
- 20%+ weekly retention
- Or LLM cost exceeds budget

9. Non-Functional Requirements

Performance

- Initial page load: <2s (mobile 3G)
- Quiz interaction: <300ms response time
- LLM generation: <5s (target <3s)
- Result page render: <1s

Reliability

- Uptime: >99.5% (Vercel SLA)
- Error rate: <1% of requests
- Graceful degradation on LLM failures

Security

- HTTPS everywhere
- No secrets in client code
- Secure session management (httpOnly cookies)

- Input validation and sanitization
- Rate limiting on API endpoints

Usability

- Mobile-first design (80% of traffic expected)
- WCAG AA compliance target
- Support for Indonesian language
- Works on Chrome, Safari, Firefox (latest 2 versions)

Scalability

- Support 1000 concurrent users (Vercel auto-scales)
- Database can handle 10K users (Supabase free tier: 50K MAU)
- Horizontal scaling path via Vercel edge functions

10. Open Questions & Risks

Open Questions

1. **Email verification:** Required or optional on registration?
2. **LLM prompt quality:** Will real-world recommendations be accurate enough?
3. **Optimal quota:** Is 2/day too little or too much?
4. **Domain:** Custom domain or vercel.app subdomain for launch?

Risks & Mitigation

Risk	Impact	Probability	Mitigation
LLM hallucination (fake venues)	High	Medium	Prompt engineering + "generic type" fallback
LLM cost explosion	High	Low	Hard rate limit (2/day), monitor daily
Low conversion (landing → quiz)	High	Medium	A/B test copy, improve value prop clarity
Poor recommendation quality	High	Medium	Iteration on system prompt, collect feedback
Slow LLM response (>5s)	Medium	Low	Set timeout, show better loading UX
User abuse (multiple accounts)	Low	Medium	Phone number + IP tracking

11. Launch Plan

Phase 1: Validation (Week 1-2)

- Build MVP (all P0 features)
- Internal testing (founder + AI)
- Deploy to staging

Phase 2: Soft Launch (Week 3)

- Invite 30-50 users from personal network
- Monitor analytics daily
- Fix critical bugs
- Collect qualitative feedback

Phase 3: Iteration (Week 4-5)

- Implement P1 features based on feedback
- Improve LLM prompt based on quality issues
- Optimize conversion funnel

Phase 4: Public Launch (Week 6+)

- Announce on social media (LinkedIn, Twitter, Instagram)
- Product Hunt launch
- Local tech community (Telegram groups, Discord)
- Organic growth via word-of-mouth

12. Success Criteria (6 Weeks Post-Launch)

Minimum Viable Success:

- 200+ registered users
- 30%+ daily active users
- 25%+ day-7 retention
- 60%+ quiz completion rate
- 40%+ recommendation click-through rate

Strong Success:

- 500+ registered users
- 40%+ daily active users
- 30%+ day-7 retention
- 75%+ quiz completion rate
- 60%+ recommendation click-through rate
- Organic virality (>0.3 viral coefficient)

Decision Point:

- If minimum viable success → Continue, add P1 features
- If strong success → Scale, consider monetization
- If below minimum → Pivot or iterate heavily

13. Appendix

Definitions

- **DAU/MAU:** Daily Active Users / Monthly Active Users (engagement metric)
- **Viral Coefficient:** # of new users referred by each existing user
- **PMF (Product-Market Fit):** When product solves real problem & users keep coming back

References

- Original PRD v1.0 (dated earlier)
- Survey results (feedback validation)
- User journey documentation
- ADR (Architecture Decision Record)

Document Status: APPROVED for development

Next Steps: Generate remaining documentation (ADR, schema, API spec) → Begin development

Change Log:

- v2.0 (Oct 5, 2026): Updated quota system (2/day, daily reset, no monetization MVP)
- v1.0 (Earlier): Initial draft with 4+6 quota system

Architecture Decision Record (ADR)

Weekend Planner MVP

Version: 2.0 (Final)

Date: October 5, 2026

Status: Approved for Implementation

Authors: Agesta (Founder) + AI Development Team

Document Purpose

This ADR documents all major technical decisions for Weekend Planner MVP, including rationale, alternatives considered, and consequences. Future developers (or AI agents) should reference this document to understand "why we built it this way."

Table of Contents

1. [System Overview](#1-system-overview)
2. [Frontend Stack](#2-frontend-stack)
3. [Backend Architecture](#3-backend-architecture)
4. [Database & Storage](#4-database--storage)
5. [LLM Integration](#5-llm-integration)
6. [Authentication](#6-authentication)
7. [Quota System](#7-quota-system)
8. [Analytics & Monitoring](#8-analytics--monitoring)
9. [Hosting & Deployment](#9-hosting--deployment)
10. [Cost Structure](#10-cost-structure)
11. [Security Considerations](#11-security-considerations)
12. [Performance Targets](#12-performance-targets)
13. [Future Migration Paths](#13-future-migration-paths)

1. System Overview

Context

Solo founder building first startup, AI-assisted development, limited budget, growth-first strategy.

High-Level Architecture



Request Flow Examples

Anonymous user generates recommendation:

1. User → Nuxt page (/quiz) → Submit quiz
2. Client → POST /api/generate → Nuxt server route
3. Server checks quota (Supabase) → Allow if <1 used
4. Server → 9router API (Hermes-combo) → Get recommendations
5. Server saves to DB → Increments quota
6. Server → Client (JSON response)
7. Client renders 5 recommendation cards

Registered user logs in:

1. User → /login → Supabase Auth (email/password or Google OAuth)
2. Supabase → Issues JWT session token
3. Client stores session (httpOnly cookie)
4. All subsequent API calls include auth header
5. Server validates JWT → Identifies user → Allows 2 quota/day

2. Frontend Stack

Decision: Nuxt 3 (Vue 3) + TypeScript + Tailwind CSS

Rationale:

• Nuxt 3 benefits:

- Server-side rendering (SSR) for landing page SEO
- File-based routing (no manual route config)
- Server routes built-in (no separate backend needed)
- Auto-imports (components, composables)
- Great TypeScript support
- Vercel zero-config deployment

• Why not Next.js:

- Founder prefers Vue syntax

- Nuxt 3 has simpler mental model (pages/ vs app/)
- No difference in performance/capabilities for this use case

- **Why not plain Vue (Vite):**

- Would need separate backend (Express/Fastify)
- More setup overhead
- SSR requires manual configuration

- **TypeScript:**

- Catch errors early
- Better IDE support for solo dev
- Easier to maintain as codebase grows

- **Tailwind CSS:**

- Fast styling (no separate CSS files)
- Mobile-first utilities
- Small bundle size (purged unused classes)

Alternatives Considered:

- Next.js 14 (React) - rejected due to preference for Vue
- SvelteKit - rejected due to smaller ecosystem
- Plain HTML + vanilla JS - rejected, too primitive for SPA

Consequences:

- ■ Fast development velocity
- ■ Built-in SSR + API routes
- ■ Easy Vercel deployment
- ■■ Vue ecosystem smaller than React (but sufficient)
- ■ Team expansion harder (fewer Vue devs than React)

3. Backend Architecture

Decision: Monolithic Nuxt Server Routes (No Separate Backend)

Rationale:

- Nuxt 3 server routes are serverless functions (on Vercel)
- Sufficient for MVP scope:
- Generate recommendations (call LLM API)
- Check/update quota (database queries)
- Handle auth callbacks
- No complex business logic requiring separate service
- Reduces operational complexity (one deployment)

Directory Structure:

server/

■ ■ ■ ■ api/

■ ■ ■ ■ generate.post.ts # POST /api/generate

■ ■ ■ ■ quota.get.ts # GET /api/quota

■ ■ ■ ■ auth/

■ ■ ■ ■ register.post.ts

■ ■ ■ ■ callback.get.ts

■ ■ ■ ■ middleware/

■ ■ ■ ■ auth.ts # Verify JWT

■ ■ ■ ■ rate-limit.ts # Additional rate limiting

■ ■ ■ ■ utils/

■ ■ ■ ■ llm.ts # Hermes-combo client

■ ■ ■ ■ db.ts # Supabase client

■ ■ ■ ■ prompt.ts # System prompt templates

When to Split Backend:

- Admin dashboard with complex operations
- Background jobs (email queues, cron)
- Real-time features (WebSocket)
- Microservices needed for scale

Alternatives Considered:

- Separate Express/Fastify backend - over-engineering for MVP
- tRPC - adds complexity, type safety not critical here

Consequences:

- ■ Simpler deployment (one codebase)
- ■ Faster development (no API contract negotiation)
- ■ Lower latency (no network hop between frontend/backend)
- ■■ Harder to scale independently (frontend/backend coupled)
- ■ Can't reuse backend for mobile app (would need to refactor)

4. Database & Storage

Decision: Supabase (PostgreSQL + Auth)

Rationale:

• Free tier generous:

- 500MB database storage
- 50,000 monthly active users
- Unlimited API requests
- Built-in auth (email, OAuth)

• Built-in features:

- Row-level security (RLS)
- Real-time subscriptions (future use)
- Auto-generated REST API
- Dashboard for debugging

• Developer experience:

- SQL-based (familiar)
- Good TypeScript support
- Local development with CLI

Schema Design:

- Users table (Supabase auth.users built-in)
- Extends with user_profiles for phone

```
CREATE TABLE user_profiles (  
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  phone TEXT NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- User quota tracking

```
CREATE TABLE user_quota (  
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  quota_used INT DEFAULT 0,  
  quota_limit INT DEFAULT 2,  
  last_reset_at TIMESTAMPTZ DEFAULT NOW(),  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Generation history

```
CREATE TABLE generations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,  
  session_id TEXT, -- for anonymous users  
  quiz_input JSONB NOT NULL,  
  recommendations JSONB NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_generations_user_id ON generations(user_id);  
CREATE INDEX idx_generations_session_id ON generations(session_id);  
CREATE INDEX idx_generations_created_at ON generations(created_at DESC);
```

-- Row-level security

```
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;  
ALTER TABLE user_quota ENABLE ROW LEVEL SECURITY;  
ALTER TABLE generations ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "Users view own profile"
```

```
ON user_profiles FOR SELECT
```

```
USING (auth.uid() = user_id);
```

```
CREATE POLICY "Users view own quota"
```

```
ON user_quota FOR ALL
```

```
USING (auth.uid() = user_id);
```

```
CREATE POLICY "Users view own generations"
```

```
ON generations FOR SELECT
```

```
USING (auth.uid() = user_id OR session_id = current_setting('app.session_id', true));
```

```
-- Daily quota reset function (called by cron)
```

```
CREATE OR REPLACE FUNCTION reset_daily_quotas()
```

```
RETURNS void AS $$
```

```
BEGIN
```

```
UPDATE user_quota
```

```
SET quota_used = 0,
```

```
last_reset_at = NOW()
```

```
WHERE last_reset_at < NOW() - INTERVAL '1 day';
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Alternatives Considered:

- Firebase - more expensive, vendor lock-in
- MongoDB Atlas - free tier too small, no built-in auth
- Local PostgreSQL - requires separate hosting, no auth

Consequences:

- ■ Zero cost for MVP
- ■ Built-in auth saves development time
- ■ Easy to migrate data out (standard PostgreSQL)
- ■■ Free tier limits (500MB, need monitoring)
- ■ Vendor dependency (but migration path exists)

5. LLM Integration

Decision: Hermes-combo via 9router API (Remote)

Rationale:

- Founder already has access to 9router endpoint
- Hermes-combo is a high-quality LLM
- Remote API simplifies deployment (no self-hosting LLM)

Integration Pattern:

```
// server/utils/llm.ts
```

```
import type { QuizInput, Recommendation } from '~/types'
```

```
const SYSTEM_PROMPT = `
```

```
Kamu adalah asisten rekomendasi weekend planner yang ahli soal tempat-tempat  
di Indonesia (kafe, tempat wisata, aktivitas, restoran, ruang publik, dll).
```

```
TUGAS:
```

```
Berdasarkan input user (mood, teman jalan, budget, radius, kota, waktu),  
berikan 5 rekomendasi tempat/aktivitas yang SPESIFIK dan MASUK AKAL untuk  
kota yang disebutkan.
```

```
ATURAN PENTING:
```

1. HANYA rekomendasikan tempat yang benar-benar kemungkinan besar eksis di kota tersebut. Jika tidak yakin, berikan rekomendasi jenis tempat umum.
2. Estimasi biaya harus realistis sebagai RANGE.
3. Sesuaikan dengan SEMUA input user.
4. Alasan harus spesifik ke input user.
5. Variasi jenis tempat jika memungkinkan.
6. JANGAN disclaimer, hanya JSON.

```
FORMAT OUTPUT (WAJIB JSON):
```

```
{
```

```

"recommendations": [
{
"name": "Nama tempat atau jenis aktivitas",
"category": "kategori singkat",
"reason": "alasan spesifik kenapa cocok",
"estimated_cost": "range biaya per orang dalam Rupiah",
"location_area": "area/lokasi umum",
"best_time": "waktu terbaik sesuai input",
"confidence": "high/medium/low"
}
]
}

```

INPUT USER:

- Mood: {mood}
- Teman jalan: {companion}
- Budget: {budget}
- Radius: {radius}
- Kota: {city}
- Waktu: {time}

`

```

export async function generateRecommendations(input: QuizInput): Promise<Recommendation[]> {
const config = useRuntimeConfig()

```

```

const userPrompt = `

```

```

Mood: ${input.mood}

```

```

Teman jalan: ${input.companion}

```

```

Budget: ${input.budget}

```

```

Radius: ${input.radius}

```

```

Kota: ${input.city}

```

```

    waktu: ${input.time || 'Fleksibel'}

```

```

`.trim()

```

```

try {
  const response = await $fetch(config.hermesApiUrl + '/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Authorization': Bearer ${config.hermesApiKey},
      'Content-Type': 'application/json',
    },
    body: {
      model: 'hermes-combo',
      messages: [
        { role: 'system', content: SYSTEM_PROMPT },
        { role: 'user', content: userPrompt }
      ],
      temperature: 0.7,
      max_tokens: 2000
    },
    timeout: 15000 // 15 second timeout
  })

  const content = response.choices[0].message.content
  const parsed = JSON.parse(content)

  return parsed.recommendations.slice(0, 5) // Ensure exactly 5

} catch (error) {
  console.error('LLM generation failed:', error)
  throw new Error('Failed to generate recommendations')
}
}

```

Error Handling:

- Timeout after 15s → Show user-friendly error
- JSON parse failure → Retry once, then fallback
- API rate limit → Should not happen (quota enforced in DB)

- Don't count failed generations against user quota

Cost Tracking:

```
// Log every LLM call for cost analysis
await supabase.from('llm_logs').insert({
  user_id: userId,
  model: 'hermes-combo',
  prompt_tokens: estimateTokens(prompt),
  cost_estimate: estimateCost(tokens),
  success: true
})
```

Alternatives Considered:

- OpenAI GPT-4o-mini - requires API key management, per-request cost
- Self-hosted LLM - too complex for solo founder
- Gemini Flash - quality unknown, similar complexity

Consequences:

- ■ High-quality recommendations (Hermes-combo proven)
- ■ No LLM hosting complexity
- ■ Cost potentially lower (depends on 9router pricing)
- ■■ Dependency on 9router availability
- ■ Need to confirm 9router pricing model

6. Authentication

Decision: Supabase Auth (Email/Password + Google OAuth)

Rationale:

- Built into Supabase (no separate auth service)
- Supports multiple providers
- JWT-based, secure by default
- Free tier: 50K MAU

Registration Flow:

// Email + Password

```
const { data, error } = await supabase.auth.signUp({
  email: email,
  password: password,
  options: {
    data: {
      phone: phone // Store phone in user metadata
    }
  }
})
```

// Create profile after signup

```
await supabase.from('user_profiles').insert({
  user_id: data.user.id,
  phone: phone
})
```

// Initialize quota

```
await supabase.from('user_quota').insert({
  user_id: data.user.id,
  quota_used: 0,
  quota_limit: 2
})
```

Google OAuth:

// Initiate OAuth

```
const { data, error } = await supabase.auth.signInWithOAuth({
  provider: 'google',
  options: {
    redirectTo: 'https://your-domain.com/auth/callback',
    queryParams: {
      access_type: 'offline',

```

```
prompt: 'consent',
}
}
})

// After OAuth callback, collect phone if first time
if (isFirstLogin) {
// Show modal to collect phone
await supabase.from('user_profiles').insert({
user_id: user.id,
phone: phone
})
}
```

Session Management:

- Session stored in httpOnly cookie (secure)
- Auto-refresh before expiry
- Logout clears session from client + server

Phone Number Requirement:

- Required field on registration
- Format validation: +62 (Indonesia)
- Not verified in MVP (SMS verification post-MVP)
- Purpose: User accountability, future SMS notifications

Email Verification:

- **Decision: Optional for MVP** (reduce friction)
- Send verification email, but allow login without
- Can add requirement post-launch if abuse detected

Alternatives Considered:

- NextAuth/Auth.js - requires more setup
- Firebase Auth - vendor lock-in, cost
- Roll own JWT - security risk, time-consuming

Consequences:

- ■ Battle-tested auth system
- ■ Multiple providers (easy to add more)
- ■ Zero setup cost
- ■■ Phone collection may reduce signup conversion
- ■ Supabase dependency

7. Quota System

Decision: 2 Quota/Day, Daily Reset, DB-Enforced

Rules:

- All users: **2 quota per day**
- Anonymous: Can use **1 quota** (must login for 2nd)
- Each generate: **5 recommendations**
- Reset: Every day at **00:00 WIB** (UTC+7)
- No free regenerates (each counts)

Implementation:

```
// server/api/generate.post.ts
export default defineEventHandler(async (event) => {
  const body = await readBody(event)
  const user = event.context.user // from auth middleware
  const sessionId = getCookie(event, 'session_id') // anonymous tracking

  // Check quota
  const quota = await checkQuota(user?.id, sessionId)

  if (!quota.allowed) {
    if (quota.reason === 'login_required') {
      return { error: 'LOGIN_REQUIRED', message: 'Login untuk generate lagi' }
    }
    if (quota.reason === 'quota_exceeded') {
```

```

return {
  error: 'QUOTA_EXCEEDED',
  message: 'Quota habis. Reset besok jam 00:00 WIB',
  reset_at: quota.resetAt
}
}
}

// Generate recommendations
const recommendations = await generateRecommendations(body)

// Save to DB
await supabase.from('generations').insert({
  user_id: user?.id,
  session_id: sessionId,
  quiz_input: body,
  recommendations: recommendations
})

// Increment quota
if (user) {
  await supabase.rpc('increment_quota', { user_id: user.id })
} else {
  // Mark anonymous session as used
  setCookie(event, 'quota_used', '1', { maxAge: 86400 }) // 24h
}

return { recommendations, quota_remaining: quota.remaining - 1 }
})

```

Quota Check Logic:

```

  async function checkQuota(userId: string | null, sessionId: string) {
    if (!userId) {
      // Anonymous user
      const quotaUsed = getCookie('quota_used')

```

```

if (quotaUsed === '1') {
  return {
    allowed: false,
    reason: 'login_required',
    remaining: 0
  }
}

return { allowed: true, remaining: 1 }
}

// Registered user
const { data } = await supabase
  .from('user_quota')
  .select('quota_used, quota_limit, last_reset_at')
  .eq('user_id', userId)
  .single()

// Check if needs reset
const now = new Date()
const lastReset = new Date(data.last_reset_at)
const hoursSinceReset = (now - lastReset) / (1000 * 60 * 60)

if (hoursSinceReset >= 24) {
  // Reset quota
  await supabase
    .from('user_quota')
    .update({ quota_used: 0, last_reset_at: now })
    .eq('user_id', userId)
  data.quota_used = 0
}

if (data.quota_used >= data.quota_limit) {
  const resetAt = new Date(lastReset)
  resetAt.setHours(resetAt.getHours() + 24)
}

```

```

return {
  allowed: false,
  reason: 'quota_exceeded',
  remaining: 0,
  resetAt: resetAt.toISOString()
}

return {
  allowed: true,
  remaining: data.quota_limit - data.quota_used
}
}

```

Scheduled Reset (Backup):

```

-- Supabase pg_cron extension

SELECT cron.schedule(
  'reset-daily-quotas',
  '0 0 *', -- Every day at 00:00 UTC
  $$
UPDATE user_quota
SET quota_used = 0, last_reset_at = NOW()
WHERE last_reset_at < NOW() - INTERVAL '1 day';
$$
);

```

Why No Free Regenerates:

- Simplifies logic (no tracking regenerate count per session)
- Encourages thoughtful quiz input
- Prevents LLM cost explosion from users spamming regenerate
- 2 quota/day already generous (4 quiz attempts over weekend)

Alternatives Considered:

- Lifetime quota (10 total) - rejected, poor retention

- 3-5 quota/day - rejected, want to start conservative
- Free regenerates (2x per quiz) - rejected, complexity + cost risk

Consequences:

- ■ Simple to implement and understand
- ■ Cost-controlled (max 2 LLM calls/user/day)
- ■ Daily reset drives return behavior
- ■■ May feel restrictive to power users (monitor feedback)
- ■ No flexibility for special cases (unless manual override)

8. Analytics & Monitoring

Decision: Google Analytics 4 (GA4)

Rationale:

- Completely free, unlimited events
- Industry standard (easy to find help/resources)
- Good dashboard for non-technical users
- Future team members likely already familiar

Events to Track:

// Page views (automatic)

pageview('/') // Landing

pageview('/quiz') // Quiz started

pageview('/result') // Result viewed

// Custom events

gtag('event', 'quiz_started')

gtag('event', 'quiz_completed', {

mood: mood,

budget: budget,

location: location,

```
has_companion: !!companion
```

```
})
```

```
gtag('event', 'recommendation_generated', {
```

```
is_logged_in: !!user,
```

```
quota_remaining: remaining,
```

```
response_time_ms: responseTime,
```

```
recommendation_count: 5
```

```
})
```

```
gtag('event', 'recommendation_clicked', {
```

```
recommendation_index: index,
```

```
category: category,
```

```
action: 'view_maps' // or 'save'
```

```
})
```

```
gtag('event', 'login_prompted', {
```

```
reason: 'quota_limit' // or 'save_favorite'
```

```
})
```

```
gtag('event', 'user_registered', {
```

```
method: 'email' // or 'google'
```

```
})
```

```
gtag('event', 'quota_exhausted', {
```

```
user_type: 'registered',
```

```
total_generations_today: 2
```

```
})
```

```
gtag('event', 'share_clicked', {
```

```
share_method: 'copy_link' // or 'whatsapp', 'instagram'
```

```
})
```

Server-Side Logging (Cost Tracking):

```
// Log to Supabase for internal analytics
```

```
await supabase.from('analytics_events').insert({
  event_type: 'llm_generation',
  user_id: userId,
  session_id: sessionId,
  metadata: {
    model: 'hermes-combo',
    response_time_ms: elapsed,
    success: true,
    cost_estimate: estimatedCost
  }
})
```

Monitoring Setup:

- **Vercel Analytics:** Built-in (Web Vitals, performance)
- **Supabase Dashboard:** Database queries, auth events
- **Custom dashboard:** Build simple Next.js app to query `analytics_events` table

Alerts:

- Vercel: Email on deployment failure
- Supabase: Email on high database usage (>400MB)
- Custom: Daily email with key metrics (registrations, generations, errors)

Alternatives Considered:

- Posthog - free tier limited (1M events), overkill for MVP
- Mixpanel - limited free tier, complex setup
- Amplitude - similar to Mixpanel
- Plausible - privacy-focused but costs \$9/month

Consequences:

- ■ Zero cost
- ■ Unlimited events
- ■ Easy setup (one script tag)
- ■■ Privacy concerns (third-party tracking)
- ■ Less powerful than Posthog (no cohort analysis, funnels limited)

9. Hosting & Deployment

Decision: Vercel (Hobby Tier)

Rationale:

- Nuxt 3 zero-config deployment
- Free tier generous:
- Unlimited deployments
- 100GB bandwidth/month
- Serverless functions included
- Preview deployments per PR
- Automatic HTTPS
- Global CDN
- Great DX (Git push → auto deploy)

Deployment Strategy:

.github/workflows/deploy.yml (optional, Vercel auto-deploys)

name: Deploy to Vercel

on:

push:

branches: [main]

pull_request:

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- uses: actions/setup-node@v3
- run: npm ci

- run: npm run build
- uses: vercel/actions@v1

with:

vercel-token: \${{ secrets.VERCEL_TOKEN }}

vercel-org-id: \${{ secrets.VERCEL_ORG_ID }}

vercel-project-id: \${{ secrets.VERCEL_PROJECT_ID }}

Branch Strategy:

- main → Production (auto-deploy)
- staging → Staging environment (auto-deploy)
- Feature branches → Preview URLs (auto-generated)

Environment Variables (Vercel Secrets):

Production

SUPABASE_URL=https://xxx.supabase.co

SUPABASE_KEY=eyJxxx...

SUPABASE_SERVICE_KEY=eyJxxx... # server-side only

HERMES_API_URL=https://9router-endpoint.com

HERMES_API_KEY=sk-...

NUXT_PUBLIC_GA_ID=G-XXXXXXXXXX

Staging (separate Supabase project)

SUPABASE_URL=https://staging-xxx.supabase.co

...

Rollback Strategy:

- Vercel keeps previous deployments
- One-click rollback in dashboard
- Or: `vercel rollback <deployment-id>`

Edge Case: LLM Endpoint Local

- If Hermes-combo is localhost only:
- Option 1: Deploy Hermes to cloud (Railway, Fly.io, Render)

- Option 2: Use VPN/tunnel (not recommended for production)
- Option 3: Fallback to OpenAI API for production

Alternatives Considered:

- Netlify - similar to Vercel, slightly less integrated with Nuxt
- Railway - good for full-stack, but Vercel better for frontend
- AWS Amplify - more complex, overkill
- Traditional VPS (DigitalOcean) - requires manual setup, not serverless

Consequences:

- ■ Zero cost for MVP
- ■ Excellent developer experience
- ■ Auto-scaling (no capacity planning needed)
- ■ Global CDN (fast worldwide)
- ■■ Vendor lock-in (but migration path exists via Docker)
- ■ Cold start latency on serverless functions (usually <1s)

10. Cost Structure

Monthly Cost Estimate (First 100 Users)

Service	Free Tier	Usage Estimate	Cost
Vercel Hosting	100GB bandwidth	~2GB (100 users × 20KB avg)	\$0
Supabase	500MB DB + 50K MAU	~50MB + 100 users	\$0
9router API (Hermes)	■ (Need confirmation)	200 requests (100 users × 2/day)	■
Google Analytics	Unlimited	Unlimited events	\$0
Domain (optional)	N/A	1 domain	~\$15/year
TOTAL			~\$0-15/year

LLM Cost Estimate (if using OpenAI as reference):

- 100 users × 2 generations/day × 30 days = 6,000 requests/month
- Avg tokens: 1,100 per request (650 prompt + 450 completion)
- GPT-4o-mini: \$0.00015/1K input + \$0.0006/1K output
- Cost: ~\$6/month for 6,000 requests
- Assuming 9router is similar or lower

Break-Even Analysis:

- If 9router costs ~\$0.001/request: ~\$6/month at 100 users
- Sustainable up to ~\$50/month = 50,000 requests = 833 daily active users
- Monetization trigger: When cost >\$50/month or 500+ registered users

Cost Monitoring:

-- Daily cost query

```
SELECT
DATE(created_at) as date,
COUNT(*) as generations,
COUNT() 0.001 as estimated_cost_usd
FROM generations
WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY DATE(created_at)
ORDER BY date DESC;
```

11. Security Considerations

API Security

```
// Rate limiting (additional layer beyond quota)
import { rateLimit } from '~/server/utils/rate-limit'

export default defineEventHandler(async (event) => {
  const ip = getRequestIP(event)
  const limited = await rateLimit(ip, { max: 10, window: 60000 }) // 10 req/min

  if (limited) {
    throw createError({ statusCode: 429, message: 'Too many requests' })
  }

  // ... rest of handler
})
```

Input Validation

```
import { z } from 'zod'

const quizSchema = z.object({
  mood: z.enum(['santai', 'aktif', 'produktif', 'quality-time', 'lainnya']),
  companion: z.enum(['sendiri', 'pasangan', 'teman', 'keluarga-kecil', 'keluarga-besar']),
  budget: z.enum(['hemat', 'menengah', 'leluasa']),
  radius: z.enum(['jabodetabek', 'road-trip', 'lainnya']),
  city: z.string().max(100).optional(),
  time: z.enum(['pagi', 'siang', 'seharian']).optional()
})

export default defineEventHandler(async (event) => {
  const body = await readBody(event)

  // Validate & sanitize
  const validated = quizSchema.parse(body)

  // ... proceed with validated data
})
```

Environment Security

- Never expose API keys to client
- Use Vercel environment variables (encrypted)
- Rotate keys if leaked
- Separate staging/production keys

Database Security

- Row-level security (RLS) enabled
- Users can only access own data
- Service key only used server-side
- Prepared statements (Supabase client handles this)

Headers

```
// nuxt.config.ts
```

```
export default defineNuxtConfig({
  nitro: {
    headers: {
      'X-Frame-Options': 'DENY',
      'X-Content-Type-Options': 'nosniff',
      'Referrer-Policy': 'strict-origin-when-cross-origin',
      'Permissions-Policy': 'geolocation=(), microphone=(), camera=()'
    }
  }
})
```

12. Performance Targets

Metrics

- **Landing page:**

- First Contentful Paint (FCP): <1.5s
- Largest Contentful Paint (LCP): <2.5s
- Time to Interactive (TTI): <3s

- **Quiz page:**

- Initial load: <1s
- Quiz interaction: <300ms

- **Generate flow:**

- LLM response: <5s (target <3s)
- Total quiz → result: <6s

- **Result page:**

- Render: <500ms

Optimization Strategies

- SSR for landing page (instant FCP)
- Lazy load non-critical components

- Image optimization (Nuxt image module)
- Minimize bundle size (<200KB initial JS)
- Use Vercel Edge Network (global CDN)

Monitoring

- Vercel Analytics (Web Vitals)
- Google Analytics (page load times)
- Custom timing events

13. Future Migration Paths

When to Migrate/Upgrade

Database (Supabase → Self-hosted PostgreSQL):

- When: >400MB data or >40K MAU
- How: pg_dump + restore to managed PostgreSQL (AWS RDS, DigitalOcean)
- Effort: Low (standard PostgreSQL)

Hosting (Vercel → Self-hosted):

- When: >90GB bandwidth/month or need more control
- How: Docker + Kubernetes or Railway
- Effort: Medium (need DevOps)

LLM (Hermes-combo → Multiple providers):

- When: 9router unavailable or quality issues
- How: Abstract LLM client, add fallback providers
- Effort: Low (already abstracted in `llm.ts`)

Auth (Supabase → Auth0 or custom):

- When: >40K MAU or need advanced features
- How: Migrate users via API, dual-run period
- Effort: High (careful user migration)

Monolith → Microservices:

- When: >10K DAU, need independent scaling
- How: Extract server routes to separate services
- Effort: High (architecture redesign)

Appendix: Decision Summary Table

Decision Area	Choice	Runner-up	Reason
Frontend	Nuxt 3	Next.js	Founder prefers Vue, simpler
Backend	Nuxt server routes	Express	No separate backend needed
Database	Supabase	Firebase	Free tier, built-in auth, PostgreSQL
LLM	Hermes-combo	OpenAI GPT-4o-mini	Already available, high quality
Auth	Supabase Auth	NextAuth	Built-in, zero setup
Quota	2/day, daily reset	Lifetime quota	Retention > monetization for MVP
Analytics	Google Analytics 4	Posthog	Free unlimited, standard tool
Hosting	Vercel	Netlify	Best Nuxt integration
Deployment	Git push → auto	Manual	Fastest iteration

Document Status: APPROVED

Next Action: Proceed to implementation (Project Init)

Review Schedule: After soft launch (30 users), review and update based on real data

User Journey - Weekend Planner MVP

Version: 2.0 (Updated for final quota system)

Date: October 5, 2026

Status: Final

Overview

Weekend Planner has 2 main user types with distinct journeys:

1. **Anonymous Users** (first-time visitors, no account)
2. **Registered Users** (logged-in)

Quota Rules:

- All users: **2 quota per day** (reset 00:00 WIB)
- Anonymous: Can use **1 quota** (must login for 2nd)
- Each generate: **5 recommendations**
- No free regenerates (each counts)

Journey 1: Anonymous User (First Visit)

Step 1: Landing Page (/)

Goal: Convert visitor to quiz starter

Elements:

- Hero headline: "Bingung Weekend Ngapain? Kami Bantu Putuskan!"
- Subheadline: "5 pertanyaan singkat → 5 rekomendasi tempat yang pas buatmu"
- Primary CTA: "Mulai Sekarang" (large button)
- Trust signals: "Gratis - 30 detik - Tanpa login dulu"
- Optional: Social proof ("1,247 orang udah dapat rekomendasi minggu ini")

User Action:

→ Click "Mulai Sekarang"

→ Navigate to /quiz

Analytics:

gtag('event', 'landing_cta_clicked')

Step 2: Quiz Page (/quiz)

Goal: Collect enough context for quality recommendations

UI Structure:

- Progress indicator (Question 2 of 5)
- One question at a time (mobile-first)
- Large tap targets (radio buttons)
- "Lanjut" button (enabled after selection)

Questions:

1. "Weekend ini lu pengen yang gimana?"

- Santai/healing ■
- Aktif/petualangan ■
- Produktif (kerja sambil enjoy) ■
- Quality time bareng orang tersayang ♥■
- Lainnya (text input)

2. "Sama siapa?"

- Sendiri ■
- Pasangan (2 orang) ■
- Teman (2-4 orang) ■
- Keluarga kecil, dengan anak ■■■■■■
- Keluarga besar ■■■■■■■■

3. "Usia anak?" (conditional, only if family with kids)

- Balita (0-5 tahun)
- SD (6-12 tahun)

- Remaja (13+ tahun)
- Tidak ada anak

4. "Budget per orang?"

- Hemat (<100rb) ■
- Menengah (100rb-300rb) ■
- Leluasa (>300rb) ■

5. "Mau kemana?"

- Sekitar Jabodetabek ■
- Jauh, mau road trip ■
- Lainnya (text input untuk kota)

6. "Waktu?" (optional)

- Pagi (sebelum 12:00) ■■
- Siang/Sore (12:00-18:00) ■■
- Sehari (pagi sampai malam) ■

User Action:

- Answer all required questions
- Click "Generate Rekomendasi"
- Navigate to loading state → /result

Analytics:

```
gtag('event', 'quiz_started')

// On each answer

gtag('event', 'quiz_question_answered', { question_num: 1 })

// On completion

gtag('event', 'quiz_completed', {
  mood: mood,
  companion: companion,
  budget: budget,
  location: location
})
```

Step 3: Loading State

Duration: 2-5 seconds (LLM API call)

UI:

- Centered spinner or progress animation
- Text: "Lagi cariin tempat yang cocok buat kamu..."
- Optional: Rotating tips
- "Pro tip: Cek jam operasional di Google Maps dulu ya"
- "Jangan lupa booking kalau weekend rame!"
- "Ajak teman untuk lebih seru 🎲"

Background Process:

1. Server receives quiz data
2. Check quota (anonymous: max 1)
3. Generate session_id (cookie)
4. Call Hermes-combo API
5. Parse JSON response
6. Save to database
7. Increment quota counter
8. Return 5 recommendations

Error Handling:

- If API fails → Show error modal, don't count quota
- If timeout (>15s) → Show retry option
- If quota exceeded → Show login prompt

Step 4: Result Page - First Generation (/result?session=xxx)

Header:

- Title: "Ini Rekomendasi Weekend Buat Kamu!"
- Quota indicator: "🎲 1x gratis sudah dipakai - Login untuk 1x lagi"

Main Content: 5 Recommendation Cards



■ ■■ Pantai Ancol ■

■ [Badge: Outdoor - Wisata] ■



■ Kenapa Cocok: ■

■ "Perfect untuk quality time bareng" ■

■ pasangan dengan suasana santai. Ada ■

■ area tenang jauh dari keramaian." ■



■ ■ Budget: Rp 150rb - 250rb/orang ■

■ ■ Area: Jakarta Utara ■

■ ■ Best time: Sabtu sore (16:00-18:00) ■ ■



■ [Button: ■ Lihat di Maps] ■

■ [Button: ■ Simpan] (login required) ■



[4 more cards...]

Actions Available:

1. **"Lihat di Maps"** - Opens Google Maps search in new tab

```
window.open(https://www.google.com/maps/search/${encodeURIComponent(name + ' ' + area)}))
```

```
gtag('event', 'recommendation_clicked', { index, action: 'maps' })
```

2. **"Simpan"** - Shows login modal (feature requires auth)

```
if (!user) {
```

```
showLoginModal()
```

```
gtag('event', 'login prompted', { reason: 'save favorite' })
```

}

3. "Generate Lagi" (sticky bottom button)

- For anonymous user → **Show login wall:**

4. "Ubah Jawaban" - Back to quiz with pre-filled answers

Path A: Satisfied, Exits

Path B: Wants More, Prompted to Login

Path C: Not Satisfied, Changes Quiz

Step 5: Auth Flow (Login or Register)

Form:

- Email (validation)
- Password
- "Remember me" checkbox
- "Lupa password?" link

Actions:

- "Login" button → Supabase Auth
- "Login dengan Google" button → OAuth flow

Option B: Register (/register)

Form:

- Email (validation, check duplicate)
- Phone number (+62 format, validation)
- Password (min 8 chars, strength indicator)
- Confirm password
- Checkbox: "Setuju dengan [Syarat] dan [Privasi]"

Actions:

- "Daftar" button → Supabase signup + create profile
- "Daftar dengan Google" button → OAuth + collect phone

Google OAuth Flow:

1. Click "Login dengan Google"
2. Redirect to Google consent screen
3. User approves
4. Redirect to /auth/callback
5. If first time:
 - Show modal: "Masukkan nomor HP"
 - Save to user_profiles
6. Initialize quota (2/day)
7. Redirect back to /quiz or previous page

Success:

- Set session cookie (httpOnly)
- Show toast: "Selamat datang! Kamu punya 1x generate lagi hari ini"
- Redirect to /quiz or /result (if came from there)

Analytics:

```
gtag('event', 'user_registered', { method: 'email' | 'google' })
gtag('event', 'user_logged_in', { method: 'email' | 'google' })
```

Journey 2: Registered User (Logged In)

Step 6: Returning User - Quiz (/quiz)

Differences from Anonymous:

Header:

- "Hai, Agesta! ■"
- Quota indicator: "Quota hari ini: 2/2 tersisa ■"

Features:

- Quiz answers auto-saved (can resume if interrupted)
- Optional: "Pakai quiz terakhir" quick action

User Action:

- Complete quiz
- Generate (quota: 1/2 remaining)

Step 7: Registered User Result Page

Header:

- "Rekomendasi Weekend Buat Agesta!"
- Quota: "Quota: 1/2 tersisa"

New Capabilities:

1. "Simpan" works (no login wall)

- Click "Simpan" → Saved to user_profiles
- Show toast: "■ Tersimpan!"
- Can view in /favorites

2. View History (nav link)

- Click "History" in navbar
- See list of past generations
- Format:

Sabtu, 5 Okt 2026 - 14:30

"Santai bareng pasangan, budget menengah, Jabodetabek"

[Lihat Lagi]

3. Generate Again

- Click "Generate Lagi" → Back to quiz
- Complete new quiz → Generate (quota: 0/2)
- After 2nd generate: **Quota exhausted**

Step 8: Quota Exhausted (2/2 Used)

Modal/Screen:

[illegible]

■ Quota Hari Ini Habis! ■ ■

□ □

■ Reset besok jam 00:00 WIB ■

■ ■ 8 jam 23 menit lagi ■

■ ■

■ Sambil nunggu, kamu bisa: ■

■ - Lihat history rekomendasi ■

■ - Share hasil ke teman ■

■ - Cek tempat yang udah kamu save ■

■ ■

■ [Lihat History] [Share] ■

[illegible]

Available Actions:

1. **"Lihat History"** → /history
2. **"Share"** → Copy link or share to WhatsApp/Instagram
3. **Wait for reset** → Come back tomorrow

Analytics:

```
gtag('event', 'quota_exhausted', {
  user_type: 'registered',
  total_generations_today: 2
})
```

Step 9: Next Day (Quota Reset)

Trigger: Automated at 00:00 WIB (database cron job)

Database Update:

```
UPDATE user_quota
SET quota_used = 0, last_reset_at = NOW()
WHERE last_reset_at < NOW() - INTERVAL '1 day';
```

User Returns:

- Sees updated quota: "Quota: 2/2 tersisa ■"
- Can generate 2 more times
- **Cycle repeats**

Navigation Structure

Anonymous User Nav

■ [Logo] Weekend Planner ■
■ [Login] [Mulai] ■

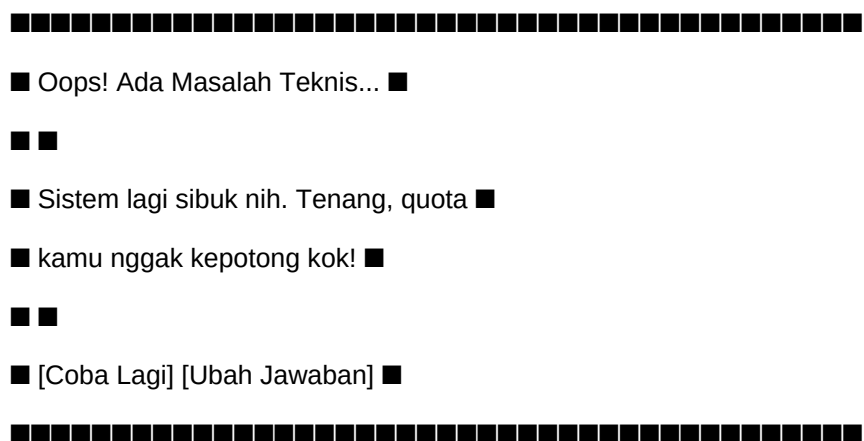
Logged-In User Nav



Edge Cases & Error Handling

Case 1: LLM Generation Failed

Error Screen:



Behavior:

- Don't count as quota used
- Allow immediate retry
- Log error to analytics

Case 2: Network Timeout (>15s)

Loading State Message:

Permintaan taking longer than expected...

[Cancel] [Keep Waiting]

After Cancel:

- Return to quiz
- Don't count quota
- Option to retry

Case 3: User Clears Browser Data

Anonymous:

- Lose session_id
- Can generate again (new anonymous session)
- Effectively "reset" quota (minor abuse vector, acceptable)

Registered:

- Session cleared → Must login again
- Quota preserved in database

Case 4: Shared Result Link

Scenario: User copies /result?session=xxx and sends to friend

Behavior:

- Friend opens link → Can view results (read-only)
- No "Simpan" or "Generate Lagi" buttons
- CTA: "Buat rekomendasi sendiri" → Landing page

Case 5: Multiple Tabs/Devices

Scenario: User opens app in 2 tabs, generates in both

Behavior:

- Quota is server-side (database)
- First tab generates → quota: 1/2
- Second tab generates → quota: 0/2
- Third attempt (either tab) → Quota exhausted

Mobile-Specific Considerations

Quiz Flow

- **One question per screen** (no scroll between questions)
- Large tap targets (min 44×44px)
- Swipe to next question (optional)

Result Cards

- **Stack vertically** (one card = full viewport width)
- Swipeable carousel (alternative layout)
- Sticky CTA at bottom ("Generate Lagi")

Modals

- Use **bottom sheet** instead of center modal
- Easier thumb reach on tall screens

Performance

- Lazy load images (if added later)
- Preload next question in quiz
- Optimistic UI (show loading immediately)

Analytics Events Summary

Page Views (Automatic)

pageview('/') // Landing

pageview('/quiz') // Quiz started

pageview('/result') // Result viewed

```
pageview('/login') // Auth page
pageview('/register')
pageview('/history')
pageview('/favorites')
```

Custom Events

// Acquisition

```
gtag('event', 'landing_cta_clicked')
gtag('event', 'quiz_started')
```

// Engagement

```
gtag('event', 'quiz_question_answered', { question_num })
gtag('event', 'quiz_completed', { mood, budget, location, companion })
gtag('event', 'recommendation_generated', {
  is_logged_in,
  quota_remaining,
  response_time_ms
})
gtag('event', 'recommendation_clicked', {
  index,
  action: 'maps' | 'save',
  category
})
```

// Auth

```
gtag('event', 'login_prompted', { reason })
gtag('event', 'user_registered', { method })
gtag('event', 'user_logged_in', { method })
```

// Retention

```
gtag('event', 'quota_exhausted', { user_type })
gtag('event', 'history_viewed')
gtag('event', 'favorite_saved')
gtag('event', 'result_shared', { method })
```

// Errors

gtag('event', 'generation_failed', { error_type })

gtag('event', 'timeout_occurred')

Success Metrics by Journey Stage

Landing Page

- **CTA Click Rate:** >40%
- **Bounce Rate:** <50%
- **Avg Time on Page:** >15 seconds

Quiz

- **Start → Complete Rate:** >70%
- **Avg Completion Time:** 30-60 seconds
- **Drop-off:** <20% per question

Result Page

- **Recommendation Click Rate:** >50% (at least 1 clicked)
- **Avg Time on Page:** >60 seconds
- **Save/Share Rate:** >10%

Auth Conversion

- **Anonymous → Register:** >25%
- **Login Prompt → Signup:** >40%

Retention

- **Day 2 Return:** >30%
- **Day 7 Return:** >20%
- **Quota Reset → Comeback:** >40%

Future Journey Enhancements (Post-MVP)

1. **Onboarding Tutorial** (first-time overlay)
2. **Quiz Presets** ("Weekend bareng pasangan lagi")
3. **Collaborative Planning** (invite friend, vote together)
4. **Social Share with OG Image** (beautiful preview card)
5. **Calendar Integration** ("Add to Google Calendar")
6. **Push Notifications** ("Weekend planner ready! ■")
7. **Referral Program** ("Ajak teman, dapat +1 quota")

Document Status: FINAL

Last Updated: October 5, 2026

Next Review: After soft launch (30 users)

Database Schema - Weekend Planner MVP

Version: 1.0

Date: October 5, 2026

Database: PostgreSQL (Supabase)

Status: Ready for Implementation

Overview

This document defines the complete database schema for Weekend Planner MVP, including tables, relationships, indexes, Row-Level Security (RLS) policies, and utility functions.

Entity Relationship Diagram (Text)

□ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □ □

■ auth.users ■ (Supabase built-in)

■ - id (PK) ■

■ - email ■

■ - created_at ■

XXXXXXXXXXXXXXXXXXXXXXXXXXXX

■ 1:1

XXXXXXXXXXXXXXXXXXXXXXXXXXXX

■ ■

▼ ▼

XX

■ user_profiles ■ ■ user_quota ■

■ - user_id (PK) ■ ■ - user_id (PK) ■

■ - phone ■■ - quota used ■

■ - created_at ■ ■ - quota_limit ■

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ - last reset ■

```
-- This table is auto-created by Supabase
-- We don't modify it directly, but reference it via foreign keys

-- Key fields we care about:
-- id: UUID (primary key)
-- email: TEXT
-- encrypted_password: TEXT (hashed)
-- email_confirmed_at: TIMESTAMPTZ
-- created_at: TIMESTAMPTZ
-- updated_at: TIMESTAMPTZ
-- last_sign_in_at: TIMESTAMPTZ
```

```
-- OAuth users also have:
-- app_metadata: JSONB (provider info)
-- user_metadata: JSONB (custom fields like phone)
```

2. user_profiles

Purpose: Extended user information beyond auth (phone, preferences)

Schema:

```
CREATE TABLE user_profiles (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  phone TEXT NOT NULL,
  phone_verified BOOLEAN DEFAULT FALSE,
  preferences JSONB DEFAULT '{}::jsonb',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

-- Indexes

```
CREATE INDEX idx_user_profiles_phone ON user_profiles(phone);
```

-- Constraints

```
ALTER TABLE user_profiles
ADD CONSTRAINT phone_format_check
CHECK (phone ~ '^+62[0-9]{9,13}$');
```

-- Comments

```
COMMENT ON TABLE user_profiles IS 'Extended user profile data';
COMMENT ON COLUMN user_profiles.phone IS 'Indonesian phone number format: +62xxxxxxxxx';
COMMENT ON COLUMN user_profiles.preferences IS 'JSON: {favorite_locations: [], notification_enabled: bool}';
```

Sample Data:

```
INSERT INTO user_profiles (user_id, phone, preferences) VALUES
(
```

```
'550e8400-e29b-41d4-a716-446655440000',  
'+628123456789',  
{'favorite_locations': ['Jakarta Selatan', 'Bogor'], 'notification_enabled': true}  
);
```

3. user_quota

Purpose: Track daily generation quota per user

Schema:

```
CREATE TABLE user_quota (  
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  quota_used INT DEFAULT 0 CHECK (quota_used >= 0),  
  quota_limit INT DEFAULT 2 CHECK (quota_limit > 0),  
  last_reset_at TIMESTAMPTZ DEFAULT NOW(),  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_user_quota_last_reset ON user_quota(last_reset_at);
```

-- Comments

```
COMMENT ON TABLE user_quota IS 'Daily generation quota tracking';  
COMMENT ON COLUMN user_quota.quota_used IS 'Number of generations used today (0-2)';  
COMMENT ON COLUMN user_quota.quota_limit IS 'Max generations per day (default 2)';  
COMMENT ON COLUMN user_quota.last_reset_at IS 'When quota was last reset to 0';
```

Sample Data:

```
INSERT INTO user_quota (user_id, quota_used, quota_limit, last_reset_at) VALUES  
(  
'550e8400-e29b-41d4-a716-446655440000', 1, 2, NOW()),  
(  
'650e8400-e29b-41d4-a716-446655440001', 2, 2, NOW() - INTERVAL '12 hours');
```

4. generations

Purpose: Store all recommendation generation history

Schema:

```
CREATE TABLE generations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,  
  session_id TEXT,  
  quiz_input JSONB NOT NULL,  
  recommendations JSONB NOT NULL,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Indexes

```
CREATE INDEX idx_generations_user_id ON generations(user_id);  
CREATE INDEX idx_generations_session_id ON generations(session_id);  
CREATE INDEX idx_generations_created_at ON generations(created_at DESC);
```

-- Composite index for user history queries

```
CREATE INDEX idx_generations_user_created ON generations(user_id, created_at DESC);
```

-- Comments

```
COMMENT ON TABLE generations IS 'All recommendation generation records';  
COMMENT ON COLUMN generations.user_id IS 'NULL for anonymous users';  
COMMENT ON COLUMN generations.session_id IS 'Anonymous tracking via cookie';  
COMMENT ON COLUMN generations.quiz_input IS 'JSON: {mood, companion, budget, radius, city, time}';  
COMMENT ON COLUMN generations.recommendations IS 'JSON array: [{name, category, reason, cost, location, time, confidence}]';
```

Sample Data:

```
INSERT INTO generations (user_id, session_id, quiz_input, recommendations) VALUES  
(  
  '550e8400-e29b-41d4-a716-446655440000',  
  NULL,  
  '{
```

```

"mood": "santai",
"companion": "pasangan",
"budget": "menengah",
"radius": "jabodetabek",
"city": null,
"time": "siang"
}::jsonb,
[
{
"name": "Taman Menteng",
"category": "outdoor",
"reason": "Cocok untuk quality time berdua dengan suasana tenang di tengah kota",
"estimated_cost": "Rp 50rb - 100rb",
"location_area": "Menteng, Jakarta Pusat",
"best_time": "Sabtu sore (16:00-18:00)",
"confidence": "high"
}
]::jsonb
);

```

5. favorites (*Post-MVP*)

Purpose: Save favorite recommendations

Schema:

```

CREATE TABLE favorites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  generation_id UUID REFERENCES generations(id) ON DELETE SET NULL,
  recommendation_index INT NOT NULL,
  recommendation_data JSONB NOT NULL,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```
);
```

```
-- Indexes
```

```
CREATE INDEX idx_favorites_user_id ON favorites(user_id);
```

```
CREATE INDEX idx_favorites_created_at ON favorites(created_at DESC);
```

```
-- Prevent duplicate favorites
```

```
CREATE UNIQUE INDEX idx_favorites_unique ON favorites(user_id, generation_id,  
recommendation_index);
```

```
-- Comments
```

```
COMMENT ON TABLE favorites IS 'User-saved favorite recommendations';
```

```
COMMENT ON COLUMN favorites.recommendation_index IS 'Which of the 5 recommendations (0-4)';
```

```
COMMENT ON COLUMN favorites.recommendation_data IS 'Snapshot of recommendation at save time';
```

6. analytics_events (*Internal Tracking*)

Purpose: Server-side event logging for cost/usage analysis

Schema:

```
CREATE TABLE analytics_events (  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
event_type TEXT NOT NULL,  
user_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,  
session_id TEXT,  
metadata JSONB DEFAULT '{}::jsonb',  
created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
-- Indexes
```

```
CREATE INDEX idx_analytics_event_type ON analytics_events(event_type);
```

```
CREATE INDEX idx_analytics_created_at ON analytics_events(created_at DESC);
```

```
CREATE INDEX idx_analytics_user_id ON analytics_events(user_id);
```

```
-- Partition by month (optional, for large scale)
```

```
-- CREATE TABLE analytics_events_2026_10 PARTITION OF analytics_events
-- FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

-- Comments

COMMENT ON TABLE analytics_events IS 'Internal analytics and cost tracking';

COMMENT ON COLUMN analytics_events.event_type IS 'e.g., llm_generation, quota_exhausted, error';

COMMENT ON COLUMN analytics_events.metadata IS 'Event-specific data: {response_time_ms,
cost_estimate, error_message}';
```

Sample Data:

```
INSERT INTO analytics_events (event_type, user_id, metadata) VALUES
(
  'llm_generation',
  '550e8400-e29b-41d4-a716-446655440000',
  '{
    "model": "hermes-combo",
    "response_time_ms": 2340,
    "prompt_tokens": 650,
    "completion_tokens": 450,
    "cost_estimate_usd": 0.0012,
    "success": true
  }::jsonb
);
```

Row-Level Security (RLS) Policies

Enable RLS on All Tables

```
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;

ALTER TABLE user_quota ENABLE ROW LEVEL SECURITY;

ALTER TABLE generations ENABLE ROW LEVEL SECURITY;

ALTER TABLE favorites ENABLE ROW LEVEL SECURITY;

ALTER TABLE analytics_events ENABLE ROW LEVEL SECURITY;
```

Policies for user_profiles

-- Users can view and update their own profile

```
CREATE POLICY "Users can view own profile"
```

```
ON user_profiles FOR SELECT
```

```
USING (auth.uid() = user_id);
```

```
CREATE POLICY "Users can update own profile"
```

```
ON user_profiles FOR UPDATE
```

```
USING (auth.uid() = user_id);
```

-- Service role can insert (during registration)

```
CREATE POLICY "Service role can insert profiles"
```

```
ON user_profiles FOR INSERT
```

```
WITH CHECK (true); -- Will be called from server with service key
```

Policies for user_quota

-- Users can view their own quota

```
CREATE POLICY "Users can view own quota"
```

```
ON user_quota FOR SELECT
```

```
USING (auth.uid() = user_id);
```

-- Server can manage quota (via service role)

```
CREATE POLICY "Service role can manage quota"
```

```
ON user_quota FOR ALL
```

```
USING (true);
```

Policies for generations

-- Users can view their own generations

```
CREATE POLICY "Users can view own generations"
```

```
ON generations FOR SELECT
USING (auth.uid() = user_id);

-- Anonymous users can view their session generations
CREATE POLICY "Anonymous can view session generations"
ON generations FOR SELECT
USING (
user_id IS NULL AND
session_id = current_setting('app.session_id', true)
);

-- Server can insert generations
CREATE POLICY "Service role can insert generations"
ON generations FOR INSERT
WITH CHECK (true);
```

Policies for favorites

```
-- Users can manage their own favorites
CREATE POLICY "Users can manage own favorites"
ON favorites FOR ALL
USING (auth.uid() = user_id);
```

Policies for analytics_events

```
-- Only service role can access (internal use only)
CREATE POLICY "Service role only"
ON analytics_events FOR ALL
USING (true);
```

Database Functions

1. Initialize New User

Purpose: Called after registration to set up profile + quota

```
CREATE OR REPLACE FUNCTION initialize_new_user(  
  p_user_id UUID,  
  p_phone TEXT  
)  
  RETURNS void AS $$  
  BEGIN  
    -- Create profile  
    INSERT INTO user_profiles (user_id, phone)  
    VALUES (p_user_id, p_phone)  
    ON CONFLICT (user_id) DO NOTHING;  
  
    -- Initialize quota  
    INSERT INTO user_quota (user_id, quota_used, quota_limit, last_reset_at)  
    VALUES (p_user_id, 0, 2, NOW())  
    ON CONFLICT (user_id) DO NOTHING;  
  END;  
  $$ LANGUAGE plpgsql SECURITY DEFINER;  
  
  COMMENT ON FUNCTION initialize_new_user IS 'Set up new user profile and quota';
```

Usage:

```
SELECT initialize_new_user('550e8400-e29b-41d4-a716-446655440000', '+628123456789');
```

2. Check and Increment Quota

Purpose: Atomic check + increment in one transaction

```
CREATE OR REPLACE FUNCTION check_and_increment_quota(  
  p_user_id UUID  
)  
  RETURNS TABLE(allowed BOOLEAN, remaining INT, reset_at TIMESTAMPTZ) AS $$
```

```

DECLARE

v_quota_used INT;

v_quota_limit INT;

v_last_reset TIMESTAMPTZ;

v_hours_since_reset NUMERIC;

BEGIN

-- Get current quota

SELECT quota_used, quota_limit, last_reset_at

INTO v_quota_used, v_quota_limit, v_last_reset

FROM user_quota

WHERE user_id = p_user_id

FOR UPDATE; -- Lock row


-- Check if needs reset (>24 hours since last reset)

v_hours_since_reset := EXTRACT(EPOCH FROM (NOW() - v_last_reset)) / 3600;


IF v_hours_since_reset >= 24 THEN

-- Reset quota

UPDATE user_quota

SET quota_used = 0, last_reset_at = NOW()

WHERE user_id = p_user_id;


v_quota_used := 0;

v_last_reset := NOW();

END IF;


-- Check if quota available

IF v_quota_used >= v_quota_limit THEN

-- Quota exhausted

RETURN QUERY SELECT

FALSE,

0,

(v_last_reset + INTERVAL '24 hours')::TIMESTAMPTZ;

ELSE

```

```

-- Increment quota

UPDATE user_quota

SET quota_used = quota_used + 1

WHERE user_id = p_user_id;

RETURN QUERY SELECT

TRUE,

v_quota_limit - v_quota_used - 1,

(v_last_reset + INTERVAL '24 hours')::TIMESTAMPTZ;

END IF;

END;

$$ LANGUAGE plpgsql SECURITY DEFINER;

COMMENT ON FUNCTION check_and_increment_quota IS 'Atomically check quota and increment if
allowed';

```

Usage:

```

SELECT * FROM check_and_increment_quota('550e8400-e29b-41d4-a716-446655440000');

-- Returns: (allowed, remaining, reset_at)

-- Example: (true, 1, '2026-10-06 00:00:00+00')

```

3. Reset Daily Quotas (Cron Job)

Purpose: Scheduled daily reset at 00:00 WIB (UTC+7)

```

CREATE OR REPLACE FUNCTION reset_daily_quotas()

RETURNS void AS $$

BEGIN

UPDATE user_quota

SET quota_used = 0,

last_reset_at = NOW()

WHERE last_reset_at < NOW() - INTERVAL '24 hours';

-- Log reset event

INSERT INTO analytics_events (event_type, metadata)

```

```
VALUES (
'quota_reset',
json_build_object(
'affected_users', (SELECT COUNT(*) FROM user_quota WHERE quota_used > 0)
)
);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

COMMENT ON FUNCTION reset_daily_quotas IS 'Reset all user quotas (called by cron)';
```

Schedule with pg_cron:

```
-- Install pg_cron extension (Supabase has it)
-- Run at 00:00 UTC (17:00 WIB, adjust to 17:00 for 00:00 WIB next day)
SELECT cron.schedule(
'reset-daily-quotas',
'0 17 *', -- 00:00 WIB = 17:00 UTC
$$SELECT reset_daily_quotas()$$
);
```

4. Get User Generation History

Purpose: Fetch paginated history for user

```
CREATE OR REPLACE FUNCTION get_user_history(
p_user_id UUID,
p_limit INT DEFAULT 10,
p_offset INT DEFAULT 0
)
RETURNS TABLE(
id UUID,
quiz_input JSONB,
recommendations JSONB,
created_at TIMESTAMPTZ
```

```

) AS $$
BEGIN
RETURN QUERY
SELECT g.id, g.quiz_input, g.recommendations, g.created_at
FROM generations g
WHERE g.user_id = p_user_id
ORDER BY g.created_at DESC
LIMIT p_limit
OFFSET p_offset;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

COMMENT ON FUNCTION get_user_history IS 'Get paginated generation history for user';

```

Usage:

```
SELECT * FROM get_user_history('550e8400-e29b-41d4-a716-446655440000', 5, 0);
```

Utility Queries

Check Quota Status

```

SELECT
u.email,
q.quota_used,
q.quota_limit,
q.last_reset_at,
(NOW() - q.last_reset_at) AS time_since_reset,
CASE
WHEN q.quota_used >= q.quota_limit THEN 'EXHAUSTED'
ELSE 'AVAILABLE'
END AS status
FROM user_quota q
JOIN auth.users u ON u.id = q.user_id
ORDER BY q.last_reset_at DESC;

```

Daily Generation Stats

```
SELECT
DATE(created_at) AS date,
COUNT(*) AS total_generations,
COUNT(DISTINCT user_id) AS unique_users,
COUNT(*) FILTER (WHERE user_id IS NULL) AS anonymous_gens,
COUNT(*) FILTER (WHERE user_id IS NOT NULL) AS registered_gens
FROM generations
WHERE created_at > NOW() - INTERVAL '7 days'
GROUP BY DATE(created_at)
ORDER BY date DESC;
```

Top Users by Generation Count

```
SELECT
u.email,
COUNT(g.id) AS total_generations,
MAX(g.created_at) AS last_generation
FROM generations g
JOIN auth.users u ON u.id = g.user_id
WHERE g.created_at > NOW() - INTERVAL '30 days'
GROUP BY u.email
ORDER BY total_generations DESC
LIMIT 10;
```

LLM Cost Estimate (Last 30 Days)

```
SELECT
DATE(created_at) AS date,
COUNT(*) AS generations,
COUNT() 0.001 AS estimated_cost_usd
```

```
FROM generations
WHERE created_at > NOW() - INTERVAL '30 days'
GROUP BY DATE(created_at)
ORDER BY date DESC;
```

Migrations

Initial Setup Script

-- Run this in Supabase SQL Editor

-- 1. Create tables

CREATE TABLE user_profiles (...); -- See above

CREATE TABLE user_quota (...);

CREATE TABLE generations (...);

CREATE TABLE favorites (...);

CREATE TABLE analytics_events (...);

-- 2. Create indexes

CREATE INDEX idx_user_profiles_phone ON user_profiles(phone);

-- ... (all indexes from above)

-- 3. Enable RLS

ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;

-- ... (all tables)

-- 4. Create policies

CREATE POLICY "Users can view own profile" ...;

-- ... (all policies)

-- 5. Create functions

CREATE OR REPLACE FUNCTION initialize_new_user(...) ...;

-- ... (all functions)

-- 6. Schedule cron job

```
SELECT cron.schedule('reset-daily-quotas', '0 17 *', ...);
```

Rollback Script

-- Drop in reverse order

```
DROP TABLE IF EXISTS analytics_events CASCADE;
```

```
DROP TABLE IF EXISTS favorites CASCADE;
```

```
DROP TABLE IF EXISTS generations CASCADE;
```

```
DROP TABLE IF EXISTS user_quota CASCADE;
```

```
DROP TABLE IF EXISTS user_profiles CASCADE;
```

```
DROP FUNCTION IF EXISTS initialize_new_user CASCADE;
```

```
DROP FUNCTION IF EXISTS check_and_increment_quota CASCADE;
```

```
DROP FUNCTION IF EXISTS reset_daily_quotas CASCADE;
```

```
DROP FUNCTION IF EXISTS get_user_history CASCADE;
```

```
SELECT cron.unschedule('reset-daily-quotas');
```

Backup & Maintenance

Daily Backup (Supabase Automatic)

- Supabase provides automatic daily backups (free tier: 7 days retention)
- Manual backup: `pg_dump` from local machine

Manual Backup

Export schema + data

```
pg_dump -h db.xxx.supabase.co \
```

```
-U postgres \
```

```
-d postgres \
```

```
-F c \
```

```
-f weekend-planner-backup-$(date +%Y%m%d).dump
```

Restore

```
pg_restore -h db.xxx.supabase.co \  
-U postgres \  
-d postgres \  
weekend-planner-backup-20261005.dump
```

Vacuum & Analyze (Automatic)

```
-- Supabase runs auto-vacuum, but can manually trigger:  
VACUUM ANALYZE generations;  
VACUUM ANALYZE analytics_events;
```

Performance Monitoring

Slow Queries

```
-- Enable pg_stat_statements extension  
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;  
  
-- Find slow queries  
SELECT  
query,  
calls,  
mean_exec_time,  
max_exec_time  
FROM pg_stat_statements  
WHERE query LIKE '%generations%'  
ORDER BY mean_exec_time DESC  
LIMIT 10;
```

Table Sizes

```
SELECT  
relname AS table_name,  
pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
```

```
pg_size_pretty(pg_relation_size(relid)) AS data_size,  
pg_size_pretty(pg_indexes_size(relid)) AS index_size  
FROM pg_catalog.pg_statio_user_tables  
ORDER BY pg_total_relation_size(relid) DESC;
```

Document Status: READY for implementation

Next Action: Execute setup script in Supabase dashboard

Review Schedule: After 1000 generations, review indexes and optimize

API Specification - Weekend Planner MVP

Version: 1.0

Date: October 5, 2026

Base URL: `https://weekend-planner.vercel.app` (Production)

Status: Final for MVP Development

Overview

Weekend Planner uses Nuxt 3 server routes for all API endpoints. All endpoints are serverless functions deployed on Vercel. Authentication uses Supabase JWT tokens passed via httpOnly cookies.

Table of Contents

1. [Authentication](#authentication)
2. [Endpoints](#endpoints)
3. [Error Handling](#error-handling)
4. [Rate Limiting](#rate-limiting)
5. [Type Definitions](#type-definitions)

Authentication

Session Management

Method: JWT tokens from Supabase Auth, stored in httpOnly cookies

Headers:

Cookie: `sb-access-token=<jwt>; sb-refresh-token=<jwt>`

Auth Middleware:

```
// server/middleware/auth.ts
```

```
export default defineEventHandler(async (event) => {
```

```
const path = event.path

// Public routes (no auth required)
const publicRoutes = ['/api/generate', '/api/health']
if (publicRoutes.includes(path)) return

// Extract token from cookie
const token = getCookie(event, 'sb-access-token')

if (!token) {
  throw createError({
    statusCode: 401,
    message: 'Unauthorized: No token provided'
  })
}

// Verify token with Supabase
const supabase = createClient()
const { data: { user }, error } = await supabase.auth.getUser(token)

  if (error || !user) {
    throw createError({
      statusCode: 401,
      message: 'Unauthorized: Invalid token'
    })
  }

// Attach user to request context
event.context.user = user
})
```

Endpoints

1. Generate Recommendations

Endpoint: POST /api/generate

Description: Generate 5 AI recommendations based on quiz input

Authentication: Optional (anonymous allowed for first generation)

Request Body:

```
{
  mood: "santai" | "aktif" | "produktif" | "quality-time" | "lainnya",
  companion: "sendiri" | "pasangan" | "teman" | "keluarga-kecil" | "keluarga-besar",
  budget: "hemat" | "menengah" | "leluasa",
  radius: "jabodetabek" | "road-trip" | "lainnya",
  city?: string, // Required if radius = "lainnya"
  time?: "pagi" | "siang" | "seharian",
  child_age?: "balita" | "sd" | "remaja" | "tidak-ada" // If companion = family
}
```

Response (Success):

```
{
  success: true,
  recommendations: [
    {
      name: "Taman Menteng",
      category: "outdoor",
      reason: "Cocok untuk quality time berdua dengan suasana tenang",
      estimated_cost: "Rp 50rb - 100rb",
      location_area: "Menteng, Jakarta Pusat",
      best_time: "Sabtu sore (16:00-18:00)",
      confidence: "high"
    }
    // ... 4 more
  ],
  quota_remaining: 1,
  is_logged_in: false
}
```

Response (Quota Exhausted - Anonymous):

```
{
  success: false,
  error: "LOGIN_REQUIRED",
  message: "Login untuk generate lagi",
  login_url: "/login"
}
```

Response (Quota Exhausted - Registered):

```
{
  success: false,
  error: "QUOTA_EXCEEDED",
  message: "Quota habis. Reset besok jam 00:00 WIB",
  reset_at: "2026-10-06T00:00:00+07:00",
  quota_remaining: 0
}
```

Response (LLM Error):

```
{
  success: false,
  error: "GENERATION_FAILED",
  message: "Gagal generate rekomendasi. Coba lagi.",
  retry_allowed: true
}
```

Status Codes:

- 200 OK - Success
- 400 Bad Request - Invalid input
- 403 Forbidden - Quota exceeded or login required
- 500 Internal Server Error - LLM generation failed
- 503 Service Unavailable - LLM service down

Implementation:

```
// server/api/generate.post.ts
```

```

import { z } from 'zod'

const quizSchema = z.object({
  mood: z.enum(['santai', 'aktif', 'produktif', 'quality-time', 'lainnya']),
  companion: z.enum(['sendiri', 'pasangan', 'teman', 'keluarga-kecil', 'keluarga-besar']),
  budget: z.enum(['hemat', 'menengah', 'leluasa']),
  radius: z.enum(['jabodetabek', 'road-trip', 'lainnya']),
  city: z.string().max(100).optional(),
  time: z.enum(['pagi', 'siang', 'seharian']).optional(),
  child_age: z.enum(['balita', 'sd', 'remaja', 'tidak-ada']).optional()
})

export default defineEventHandler(async (event) => {
  const body = await readBody(event)
  const user = event.context.user

  // Validate input
  const validated = quizSchema.parse(body)

  // Check quota
  const sessionId = getCookie(event, 'session_id') || generateSessionId()
  const quota = await checkQuota(user?.id, sessionId)

  if (!quota.allowed) {
    if (quota.reason === 'login_required') {
      throw createError({
        statusCode: 403,
        data: {
          error: 'LOGIN_REQUIRED',
          message: 'Login untuk generate lagi',
          login_url: '/login'
        }
      })
    }
  }
}

```

```
if (quota.reason === 'quota_exceeded') {
  throw createError({
    statusCode: 403,
    data: {
      error: 'QUOTA_EXCEEDED',
      message: 'Quota habis. Reset besok jam 00:00 WIB',
      reset_at: quota.resetAt
    }
  })
}

// Generate recommendations
const recommendations = await generateRecommendations(validated)

// Save to database
const generation = await saveGeneration({
  user_id: user?.id,
  session_id: sessionId,
  quiz_input: validated,
  recommendations
})

// Increment quota
await incrementQuota(user?.id, sessionId)

// Log analytics
await logEvent({
  event_type: 'llm_generation',
  user_id: user?.id,
  session_id: sessionId,
  metadata: {
    response_time_ms: responseTime,
    success: true
  }
})
```

```
}  
})  
  
return {  
  success: true,  
  recommendations,  
  quota_remaining: quota.remaining - 1,  
  is_logged_in: !!user,  
  generation_id: generation.id  
}  
})
```

2. Check Quota Status

Endpoint: GET /api/quota

Description: Get current quota status for user

Authentication: Optional

Query Parameters: None

Response (Anonymous):

```
{  
  quota_used: 0,  
  quota_limit: 1,  
  quota_remaining: 1,  
  is_logged_in: false,  
  reset_at: null // No reset for anonymous  
}
```

Response (Registered):

```
{  
  quota_used: 1,  
  quota_limit: 2,
```

```
quota_remaining: 1,  
is_logged_in: true,  
reset_at: "2026-10-06T00:00:00+07:00",  
hours_until_reset: 18.5  
}
```

Status Codes:

- 200 OK - Success

Implementation:

```
// server/api/quota.get.ts  
export default defineEventHandler(async (event) => {  
  const user = event.context.user  
  
  if (!user) {  
    // Anonymous user  
    const sessionId = getCookie(event, 'session_id')  
    const quotaUsed = getCookie(event, 'quota_used') === '1' ? 1 : 0  
  
    return {  
      quota_used: quotaUsed,  
      quota_limit: 1,  
      quota_remaining: 1 - quotaUsed,  
      is_logged_in: false,  
      reset_at: null  
    }  
  }  
  
  // Registered user  
  const quota = await getQuotaStatus(user.id)  
  
  return {  
    quota_used: quota.quota_used,  
    quota_limit: quota.quota_limit,  
    quota_remaining: quota.quota_limit - quota.quota_used,  
  }  
})
```

```
is_logged_in: true,  
reset_at: calculateResetTime(quota.last_reset_at),  
hours_until_reset: calculateHoursUntilReset(quota.last_reset_at)  
}  
})
```

3. Get Generation History

Endpoint: GET /api/history

Description: Fetch user's past generations

Authentication: Required

Query Parameters:

?limit=10&offset=0

Response:

```
{  
  generations: [  
    {  
      id: "uuid",  
      quiz_input: {  
        mood: "santai",  
        companion: "pasangan",  
        budget: "menengah",  
        radius: "jabodetabek",  
        time: "siang"  
      },  
      recommendations: [...], // Array of 5 recs  
      created_at: "2026-10-05T10:30:00Z"  
    }  
    // ... more  
  ],
```

```
total: 25,  
has_more: true  
}
```

Status Codes:

- 200 OK - Success
- 401 Unauthorized - Not logged in

Implementation:

```
// server/api/history.get.ts  
  
export default defineEventHandler(async (event) => {  
  const user = event.context.user  
  
  if (!user) {  
    throw createError({  
      statusCode: 401,  
      message: 'Login required'  
    })  
  }  
  
  const query = getQuery(event)  
  const limit = parseInt(query.limit as string) || 10  
  const offset = parseInt(query.offset as string) || 0  
  
  const { data, total } = await getGenerationHistory(user.id, limit, offset)  
  
  return {  
    generations: data,  
    total,  
    has_more: total > (offset + limit)  
  }  
})
```

4. Save Favorite

Endpoint: POST /api/favorites

Description: Save a recommendation to favorites

Authentication: Required

Request Body:

```
{  
  generation_id: "uuid",  
  recommendation_index: 0, // 0-4  
  notes?: "Pengen coba ini weekend depan"  
}
```

Response:

```
{  
  success: true,  
  favorite_id: "uuid",  
  message: "Rekomendasi berhasil disimpan"  
}
```

Status Codes:

- 200 OK - Success
- 400 Bad Request - Invalid generation_id or index
- 401 Unauthorized - Not logged in
- 409 Conflict - Already favorited

Implementation:

```
// server/api/favorites.post.ts  
  
export default defineEventHandler(async (event) => {  
  const user = event.context.user  
  
  if (!user) {  
    throw createError({ statusCode: 401, message: 'Login required' })  
  }  
  
  const { generation_id, recommendation_index, notes } = await readBody(event)
```

```

// Fetch generation

const generation = await getGeneration(generation_id)

    if (!generation || generation.user_id !== user.id) {
throw createError({ statusCode: 404, message: 'Generation not found' })
}

    if (recommendation_index < 0 || recommendation_index > 4) {
throw createError({ statusCode: 400, message: 'Invalid index' })
}

// Save favorite

const favorite = await saveFavorite({
user_id: user.id,
generation_id,
recommendation_index,
recommendation_data: generation.recommendations[recommendation_index],
notes
})

return {
success: true,
favorite_id: favorite.id,
message: 'Rekomendasi berhasil disimpan'
}
})

```

5. Get Favorites

Endpoint: GET /api/favorites

Description: Fetch user's saved favorites

Authentication: Required

Query Parameters:

?limit=20&offset=0

Response:

```
{
  favorites: [
    {
      id: "uuid",
      recommendation: {
        name: "Taman Menteng",
        category: "outdoor",
        reason: "...",
        estimated_cost: "Rp 50rb - 100rb",
        location_area: "Menteng, Jakarta Pusat",
        best_time: "Sabtu sore"
      },
      notes: "Pengen coba ini weekend depan",
      created_at: "2026-10-05T11:00:00Z"
    }
  ]
  // ... more
},
total: 8,
has_more: false
}
```

Status Codes:

- 200 OK - Success
- 401 Unauthorized - Not logged in

6. Register User

Endpoint: POST /api/auth/register

Description: Create new user account

Authentication: None

Request Body:

```
{  
  email: "agesta@example.com",  
  password: "SecurePass123!",  
  phone: "+628123456789"  
}
```

Response:

```
{  
  success: true,  
  user: {  
    id: "uuid",  
    email: "agesta@example.com"  
  },  
  message: "Registrasi berhasil"  
}
```

Status Codes:

- 201 Created - Success
- 400 Bad Request - Invalid input
- 409 Conflict - Email already exists

Implementation:

```
// server/api/auth/register.post.ts  
  
export default defineEventHandler(async (event) => {  
  const { email, password, phone } = await readBody(event)  
  
  // Validate  
  if (!email || !password || !phone) {  
    throw createError({ statusCode: 400, message: 'Missing fields' })  
  }  
  
  if (!/^\+62[0-9]{9,13}$/.test(phone)) {
```

```
throw createError({ statusCode: 400, message: 'Invalid phone format' })
}

// Create user in Supabase
const supabase = createServiceClient()
const { data, error } = await supabase.auth.signUp({
  email,
  password,
  options: {
    data: { phone }
  }
})

if (error) {
  throw createError({ statusCode: 409, message: error.message })
}

// Initialize user profile & quota
await initializeNewUser(data.user.id, phone)

// Set session cookie
setCookie(event, 'sb-access-token', data.session.access_token, {
  httpOnly: true,
  secure: true,
  maxAge: 3600
})

return {
  success: true,
  user: {
    id: data.user.id,
    email: data.user.email
  },
  message: 'Registrasi berhasil'
}
```

}}

7. Login User

Endpoint: POST /api/auth/login

Description: Authenticate existing user

Authentication: None

Request Body:

```
{  
  email: "agesta@example.com",  
  password: "SecurePass123!"  
}
```

Response:

```
{  
  success: true,  
  user: {  
    id: "uuid",  
    email: "agesta@example.com"  
  },  
  message: "Login berhasil"  
}
```

Status Codes:

- 200 OK - Success
- 401 Unauthorized - Invalid credentials

8. Logout User

Endpoint: POST /api/auth/logout

Description: End user session

Authentication: Required

Request Body: None

Response:

```
{  
  success: true,  
  message: "Logout berhasil"  
}
```

Status Codes:

- 200 OK - Success

9. Health Check

Endpoint: GET /api/health

Description: Service health status

Authentication: None

Response:

```
{  
  status: "ok",  
  timestamp: "2026-10-05T06:03:13.666Z",  
  services: {  
    database: "ok",  
    llm: "ok"  
  }  
}
```

Status Codes:

- 200 OK - All services healthy
- 503 Service Unavailable - Degraded

Error Handling

Standard Error Response

All errors follow this format:

```
{
  statusCode: 400 | 401 | 403 | 404 | 409 | 500 | 503,
  message: "Human-readable error message",
  error?: "ERROR_CODE", // Optional machine-readable code
  details?: any // Optional additional context
}
```

Error Codes

Code	HTTP Status	Meaning
VALIDATION_ERROR	400	Invalid input data
UNAUTHORIZED	401	Not logged in
LOGIN_REQUIRED	403	Anonymous quota exhausted
QUOTA_EXCEEDED	403	Registered quota exhausted
NOT_FOUND	404	Resource doesn't exist
ALREADY_EXISTS	409	Duplicate resource
GENERATION_FAILED	500	LLM error
SERVICE_UNAVAILABLE	503	External service down

Rate Limiting

Strategy

Per-IP Rate Limit: 60 requests per minute (additional layer beyond quota)

Implementation:

```
// server/middleware/rate-limit.ts
import { RateLimiterMemory } from 'rate-limiter-flexible'
```

```

const rateLimiter = new RateLimiterMemory({
  points: 60, // requests
  duration: 60 // per 60 seconds
})

export default defineEventHandler(async (event) => {
  const ip = getRequestIP(event, { xForwardedFor: true })

  try {
    await rateLimiter.consume(ip)
  } catch {
    throw createError({
      statusCode: 429,
      message: 'Too many requests. Please try again later.'
    })
  }
})

```

Response (Rate Limited):

```

{
  statusCode: 429,
  message: "Too many requests. Please try again later.",
  retry_after: 30 // seconds
}

```

Type Definitions

TypeScript Types

```

// types/quiz.ts

export type Mood = 'santai' | 'aktif' | 'produktif' | 'quality-time' | 'lainnya'
export type Companion = 'sendiri' | 'pasangan' | 'teman' | 'keluarga-kecil' | 'keluarga-besar'
export type Budget = 'hemat' | 'menengah' | 'leluasa'

```

```
export type Radius = 'jabodetabek' | 'road-trip' | 'lainnya'
export type Time = 'pagi' | 'siang' | 'seharian'
export type ChildAge = 'balita' | 'sd' | 'remaja' | 'tidak-ada'
```

```
export interface QuizInput {
  mood: Mood
  companion: Companion
  budget: Budget
  radius: Radius
  city?: string
  time?: Time
  child_age?: ChildAge
}
```

```
export interface Recommendation {
  name: string
  category: string
  reason: string
  estimated_cost: string
  location_area: string
  best_time: string
  confidence: 'high' | 'medium' | 'low'
}
```

```
export interface GenerateResponse {
  success: boolean
  recommendations?: Recommendation[]
  quota_remaining?: number
  is_logged_in?: boolean
  generation_id?: string
  error?: string
  message?: string
  reset_at?: string
}
```

```
export interface QuotaStatus {  
  quota_used: number  
  quota_limit: number  
  quota_remaining: number  
  is_logged_in: boolean  
  reset_at: string | null  
  hours_until_reset?: number  
}
```

Testing

Example cURL Requests

Generate Recommendation (Anonymous):

```
curl -X POST https://weekend-planner.vercel.app/api/generate \  
-H "Content-Type: application/json" \  
-d '{  
  "mood": "santai",  
  "companion": "pasangan",  
  "budget": "menengah",  
  "radius": "jabodetabek",  
  "time": "siang"  
}'
```

Check Quota:

```
curl https://weekend-planner.vercel.app/api/quota
```

Register:

```
curl -X POST https://weekend-planner.vercel.app/api/auth/register \  
-H "Content-Type: application/json" \  
-d '{  
  "email": "test@example.com",  
  "password": "TestPass123!",
```

```
"phone": "+628123456789"  
'
```

API Versioning

Current: No versioning (MVP)

Future: When breaking changes needed:

- Add /api/v2/ prefix
- Maintain v1 for 6 months
- Deprecation warnings in headers

Document Status: FINAL

Last Updated: October 5, 2026

Next Review: After soft launch feedback

MVP Feature Checklist - Weekend Planner

Version: 1.0

Date: October 5, 2026

Status: Ready for Development

Owner: Agesta (Solo Founder)

Overview

This checklist breaks down Weekend Planner MVP into actionable tasks with priority levels and time estimates. Use this as your development roadmap.

Priority Levels:

- **P0:** Must-have for launch (blocking)
- **P1:** Should-have (important but can defer)
- **P2:** Nice-to-have (post-launch)

Time Estimates: Based on solo dev with AI assistance

Phase 1: Project Setup & Foundation

Goal: Initialize project, configure tools, set up infrastructure

1.1 Project Initialization

- ☐ **P0** Create GitHub repository (public/private) - 15 min
- ☐ **P0** Initialize Nuxt 3 project (`npx nuxi@latest init weekend-planner`) - 10 min
- ☐ **P0** Install dependencies: TypeScript, Tailwind CSS, ESLint, Prettier - 20 min
- ☐ **P0** Configure `nuxt.config.ts` (modules, runtimeConfig) - 15 min
- ☐ **P0** Set up directory structure (pages/, components/, server/, types/) - 10 min
- ☐ **P0** Create `.env.example` file with all required env vars - 10 min

Subtotal: ~1.5 hours

1.2 Supabase Setup

- [] **P0** Create Supabase project - 5 min
- [] **P0** Run database schema SQL (from 04-Database-Schema.md) - 15 min
- [] **P0** Enable Row-Level Security policies - 10 min
- [] **P0** Create database functions (initialize_new_user, check_quota, etc.) - 20 min
- [] **P0** Configure Google OAuth in Supabase Auth settings - 10 min
- [] **P0** Test database connection from local - 10 min
- [] **P0** Schedule pg_cron job for daily quota reset - 10 min

Subtotal: ~1.5 hours

1.3 Hermes-combo / 9router Integration

- [] **P0** Verify 9router API endpoint is accessible - 5 min
- [] **P0** Test LLM generation with sample prompt - 15 min
- [] **P0** Create LLM client utility (server/utis/llm.ts) - 30 min
- [] **P0** Create system prompt template - 20 min
- [] **P0** Test full quiz → LLM → parse JSON flow - 20 min

Subtotal: ~1.5 hours

1.4 Deployment Setup

- [] **P0** Connect GitHub repo to Vercel - 5 min
- [] **P0** Configure Vercel environment variables - 10 min
- [] **P0** Set up staging environment (separate branch) - 10 min
- [] **P0** Test auto-deploy on push - 5 min
- [] **P1** Set up custom domain (optional) - 30 min

Subtotal: ~1 hour

Phase 1 Total: ~5.5 hours

Phase 2: Core Features - Backend API

Goal: Build all server routes and business logic

2.1 Authentication Endpoints

- [] **P0** POST /api/auth/register - Email/password signup - *1 hour*
 - Validate email, password, phone format
 - Call Supabase Auth signup
 - Initialize user profile + quota
 - Set session cookie
 - Error handling (duplicate email, weak password)
- [] **P0** POST /api/auth/login - Email/password login - *45 min*
 - Validate credentials
 - Call Supabase Auth login
 - Set session cookie
 - Error handling (invalid credentials)
- [] **P0** POST /api/auth/logout - End session - *15 min*
 - Clear session cookie
 - Call Supabase signOut
- [] **P0** GET /api/auth/callback - OAuth callback handler - *45 min*
 - Handle Google OAuth redirect
 - Check if first-time user
 - Collect phone if needed
 - Initialize profile + quota
- [] **P0** Auth middleware (server/middleware/auth.ts) - *30 min*
 - Extract JWT from cookie
 - Verify with Supabase
 - Attach user to request context

Subtotal: ~3.5 hours

2.2 Quota System

- [] **P0** GET /api/quota - Check quota status - 45 min
 - Anonymous: check session cookie
 - Registered: query database
 - Calculate hours until reset
 - Return status
- [] **P0** Quota check utility (server/utls/quota.ts) - 1.5 hours
 - checkQuota(userId, sessionId) function
 - Handle anonymous vs registered logic
 - Auto-reset if >24h since last reset
 - Return {allowed, remaining, resetAt, reason}
- [] **P0** Increment quota utility - 30 min
 - For registered: call check_and_increment_quota() DB function
 - For anonymous: set cookie

Subtotal: ~2.5 hours

2.3 Generation Endpoint

- [] **P0** POST /api/generate - Main generation endpoint - 2.5 hours
 - Validate quiz input with Zod
 - Check quota (fail early if exhausted)
 - Generate session_id for anonymous
 - Call LLM API with timeout (15s)
 - Parse JSON response
 - Validate 5 recommendations returned
 - Save to generations table
 - Increment quota
 - Log analytics event
 - Error handling (LLM timeout, parse failure, quota exceeded)

- [] **P0** LLM generation utility improvements - *1 hour*
- Retry logic (1 retry on timeout)
- Better error messages
- Token counting estimation
- Cost tracking

Subtotal: ~3.5 hours

2.4 History & Favorites

- [] **P1** GET /api/history - Fetch generation history - *1 hour*
- Require auth
- Paginated results (limit, offset)
- Return quiz_input + recommendations + created_at
- [] **P1** POST /api/favorites - Save favorite - *1 hour*
- Require auth
- Validate generation_id and index
- Insert into favorites table
- Handle duplicate (409 Conflict)
- [] **P1** GET /api/favorites - Fetch favorites - *45 min*
- Require auth
- Paginated results
- Return recommendation data + notes
- [] **P1** DELETE /api/favorites/:id - Remove favorite - *30 min*

Subtotal: ~3 hours

2.5 Utility Endpoints

- [] **P0** GET /api/health - Health check - *15 min*
- Check database connection
- Check LLM endpoint

- Return service status
- [] **P0** Rate limiting middleware - *45 min*
- Install rate-limiter-flexible
- 60 req/min per IP
- Return 429 on limit

Subtotal: ~1 hour

Phase 2 Total: ~13.5 hours

Phase 3: Core Features - Frontend UI

Goal: Build all user-facing pages and components

3.1 Design System & Components

- [] **P0** Tailwind config (colors, fonts, breakpoints) - *30 min*
- [] **P0** Create base components - *2 hours*
- Button (primary, secondary, ghost variants)
- Input (text, password, with validation states)
- Card
- Modal/Dialog
- Loading spinner
- Toast notification
- [] **P0** Create layout component (layouts/default.vue) - *45 min*
- Navbar (logo, login/register, user menu)
- Footer (links, copyright)
- Mobile responsive

Subtotal: ~3 hours

3.2 Landing Page

- [] **P0** pages/index.vue - Landing page - 2 hours
- Hero section with headline + CTA
- Value proposition (3 benefits)
- How it works (3 steps)
- Social proof section (optional)
- Footer
- Mobile responsive
- SEO meta tags

Subtotal: ~2 hours

3.3 Quiz Flow

- [] **P0** pages/quiz.vue - Quiz page - 3 hours
- Multi-step form (1 question per screen)
- Progress indicator
- Question components:
- Mood selector (radio buttons with icons)
- Companion selector
- Budget selector
- Location selector (with conditional text input)
- Time selector (optional)
- Child age (conditional)
- Form validation
- "Lanjut" / "Kembali" navigation
- Submit → Call /api/generate
- Loading state during generation
- Error handling (show retry)
- [] **P0** Quiz state management (composable) - 1 hour
- useQuiz() composable
- Store answers in reactive state
- Persist to localStorage (resume if refresh)

- Validation logic

Subtotal: ~4 hours

3.4 Result Page

- [] **P0** pages/result.vue - Result display - 3 hours
- Header with quota indicator
- 5 recommendation cards:
 - Name + category badge
 - Reason (personalized)
 - Cost estimate
 - Location area
 - Best time
- "Lihat di Maps" button (external link)
- "Simpan" button (login wall for anonymous)
- "Generate Lagi" sticky bottom button
- "Ubah Jawaban" link
- Mobile: cards stack vertically
- Tablet/Desktop: 2-column grid
- [] **P0** Recommendation card component - 1 hour
- Reusable card component
- Category badge styling
- Icon integration
- Responsive layout

Subtotal: ~4 hours

3.5 Authentication Pages

- [] **P0** pages/login.vue - Login page - 1.5 hours
- Email + password form
- "Remember me" checkbox

- "Lupa password?" link (stub for now)
 - "Login dengan Google" button
 - Link to register page
 - Form validation
 - Error display (invalid credentials)
 - Redirect after success
-
- [] **P0** pages/register.vue - Register page - *1.5 hours*
 - Email, phone, password, confirm password fields
 - Phone validation (+62 format)
 - Password strength indicator
 - Terms checkbox
 - "Daftar dengan Google" button
 - Form validation
 - Error display (email exists, weak password)
 - Redirect after success
-
- [] **P0** pages/auth/callback.vue - OAuth callback - *1 hour*
 - Handle OAuth redirect
 - Show phone collection modal (if first-time Google login)
 - Loading state
 - Redirect to previous page or quiz

Subtotal: ~4 hours

3.6 User Features (Post-MVP P1)

- [] **P1** pages/history.vue - Generation history - *2 hours*
 - List past generations
 - Show quiz input summary
 - "Lihat Lagi" button to view results
 - Pagination
 - Empty state
-
- [] **P1** pages/favorites.vue - Saved favorites - *2 hours*

- Grid of favorite cards
 - Notes display
 - "Hapus" button
 - Empty state
-
- [] **P1** pages/profile.vue - User profile - *1.5 hours*
 - Display email, phone
 - Edit profile (phone only)
 - Logout button

Subtotal (P1): ~5.5 hours

3.7 Error & Edge Case Pages

- [] **P0** pages/quota-exhausted.vue - Quota modal/page - *1 hour*
 - Countdown to reset
 - Alternative actions (history, share)
 - CTA to return tomorrow
-
- [] **P0** Error components - *1 hour*
 - 404 page
 - 500 error page
 - Network error modal
 - LLM generation failed modal

Subtotal: ~2 hours

Phase 3 Total (P0 only): ~19.5 hours

Phase 3 Total (with P1): ~25 hours

Phase 4: State Management & Data Flow

Goal: Connect frontend to backend, handle auth state

4.1 Composables

- [] **P0** composables/useAuth.ts - Auth state - *2 hours*
- user reactive ref
- login(), logout(), register() functions
- Auto-refresh token
- Persist state across page loads

- [] **P0** composables/useQuota.ts - Quota state - *1 hour*
- quotaStatus reactive ref
- fetchQuota() function
- Auto-update after generation
- Countdown timer to reset

- [] **P0** composables/useGenerate.ts - Generation flow - *1.5 hours*
- generate(quizInput) function
- Loading state
- Error handling
- Store result in state

Subtotal: ~4.5 hours

4.2 API Client

- [] **P0** utils/api.ts - API client wrapper - *1.5 hours*
- Fetch wrapper with auth header injection
- Error parsing
- TypeScript types
- Retry logic (optional)

Subtotal: ~1.5 hours

Phase 4 Total: ~6 hours

Phase 5: Analytics & Monitoring

Goal: Track user behavior and system health

5.1 Google Analytics Setup

- [] **P0** Install @nuxtjs/google-analytics or manual gtag - *15 min*
- [] **P0** Configure GA4 property - *15 min*
- [] **P0** Add gtag to app.vue - *15 min*
- [] **P0** Implement event tracking - *2 hours*
- Page views (automatic)
- quiz_started
- quiz_completed
- recommendation_generated
- recommendation_clicked
- login_prompted
- user_registered
- quota_exhausted
- generation_failed

Subtotal: ~3 hours

5.2 Server-Side Logging

- [] **P0** Log LLM requests to analytics_events - *30 min*
- [] **P0** Log errors to analytics_events - *30 min*
- [] **P1** Create internal dashboard page (view logs) - *2 hours*

Subtotal (P0): ~1 hour

Subtotal (with P1): ~3 hours

Phase 5 Total (P0): ~4 hours

Phase 6: Testing & Quality Assurance

Goal: Ensure everything works before launch

6.1 Manual Testing

- [] **P0** Test full anonymous flow (landing → quiz → result) - 30 min
- [] **P0** Test full registered flow (register → quiz → result × 2) - 30 min
- [] **P0** Test quota exhaustion scenarios - 30 min
- [] **P0** Test error scenarios (LLM timeout, invalid input) - 30 min
- [] **P0** Test OAuth flow (Google login) - 15 min
- [] **P0** Test mobile responsive (iPhone, Android) - 30 min
- [] **P0** Test on multiple browsers (Chrome, Safari, Firefox) - 30 min

Subtotal: ~3 hours

6.2 Automated Testing (Optional P1)

- [] **P1** Set up Vitest for unit tests - 1 hour
- [] **P1** Write tests for utility functions (quota check, LLM client) - 2 hours
- [] **P1** Set up Playwright for E2E tests - 1 hour
- [] **P1** Write E2E test for happy path - 2 hours

Subtotal (P1): ~6 hours

6.3 Bug Fixes & Polish

- [] **P0** Fix bugs found during testing - 4 hours (estimate)
- [] **P0** Polish UI/UX rough edges - 2 hours
- [] **P0** Optimize performance (lazy loading, code splitting) - 2 hours

Subtotal: ~8 hours

Phase 6 Total (P0): ~11 hours

Phase 7: Pre-Launch Preparation

Goal: Final checks and launch readiness

7.1 Content & Copy

- [] **P0** Write all UI copy (headlines, CTAs, error messages) - *1.5 hours*
- [] **P0** Review Indonesian language consistency - *30 min*
- [] **P0** Create social share OG image - *1 hour*

Subtotal: ~3 hours

7.2 SEO & Meta Tags

- [] **P0** Set up SEO meta tags (title, description) - *30 min*
- [] **P0** Add Open Graph tags - *30 min*
- [] **P0** Create robots.txt - *10 min*
- [] **P0** Create sitemap.xml - *20 min*

Subtotal: ~1.5 hours

7.3 Legal & Compliance

- [] **P1** Create Terms of Service page (simple version) - *1 hour*
- [] **P1** Create Privacy Policy page (simple version) - *1 hour*
- [] **P1** Add cookie notice (optional for MVP) - *30 min*

Subtotal (P1): ~2.5 hours

7.4 Launch Checklist

- [] **P0** Set up production environment variables - *15 min*
- [] **P0** Test production build locally - *15 min*
- [] **P0** Deploy to production - *10 min*
- [] **P0** Smoke test in production - *30 min*

- [] **P0** Set up error monitoring (Sentry optional) - *1 hour*
- [] **P0** Prepare launch announcement (social media posts) - *1 hour*

Subtotal: ~3 hours

Phase 7 Total (P0): ~7.5 hours

Time Summary

Phase	P0 (Must-Have)	P1 (Should-Have)	Total
1. Setup & Foundation	5.5h	0.5h	6h
2. Backend API	10.5h	3h	13.5h
3. Frontend UI	19.5h	5.5h	25h
4. State Management	6h	0h	6h
5. Analytics	4h	2h	6h
6. Testing & QA	11h	6h	17h
7. Pre-Launch	7.5h	2.5h	10h
TOTAL	64h	19.5h	83.5h

Development Timeline

Aggressive (Focus Mode)

- **8 hours/day** → **8 days** (P0 only)
- **8 hours/day** → **10.5 days** (P0 + P1)

Realistic (Solo Founder)

- **4-5 hours/day** → **13-16 days** (P0 only)
- **4-5 hours/day** → **17-21 days** (P0 + P1)

Recommended Schedule

- Week 1:** Phase 1-2 (Setup + Backend)
- Week 2:** Phase 3-4 (Frontend + State)
- Week 3:** Phase 5-7 (Analytics + Testing + Launch)

Risk Mitigation

High-Risk Tasks (Likely to Take Longer)

1. **LLM Integration** (Phase 1.3 + 2.3)

- Risk: API reliability, prompt tuning, JSON parsing failures
- Mitigation: Budget 2x time, implement robust error handling

2. **Quiz UX** (Phase 3.3)

- Risk: Form flow confusing, drop-off rate high
- Mitigation: Test with 3-5 users early, iterate on flow

3. **OAuth Integration** (Phase 2.1 + 3.5)

- Risk: Callback redirect issues, phone collection friction
- Mitigation: Test thoroughly, prepare fallback to email-only

4. **Mobile Responsive** (All frontend)

- Risk: Layout breaks on small screens
- Mitigation: Mobile-first design from start, test on real devices

Post-Launch Iterations

Week 1 After Launch

- [] Monitor analytics daily
- [] Collect user feedback (survey or interviews)
- [] Fix critical bugs (P0)
- [] Iterate on LLM prompt based on quality feedback

Week 2-4 After Launch

- [] Implement P1 features (history, favorites, profile)
- [] A/B test landing page copy
- [] Optimize conversion funnel (landing → quiz → result)
- [] Add missing features based on user requests

Feature Toggles (Optional)

Consider feature flags for:

- ☐ Google OAuth (can disable if issues arise)
- ☐ Phone number requirement (can make optional)
- ☐ Quota limit (can increase from 2 to 3 if LLM cost allows)
- ☐ P1 features (can enable gradually)

Success Criteria (2 Weeks Post-Launch)

Minimum Viable Success:

- ■ 50+ registered users
- ■ 60%+ quiz completion rate
- ■ 40%+ recommendation click-through rate
- ■ 20%+ day-2 return rate
- ■ <5% error rate

Strong Success:

- ■ 100+ registered users
- ■ 75%+ quiz completion rate
- ■ 60%+ recommendation click-through rate
- ■ 30%+ day-2 return rate
- ■ Organic word-of-mouth (>0.2 viral coefficient)

Document Status: READY for execution

Last Updated: October 5, 2026

Owner: Agesta

Next Action: Begin Phase 1 (Project Setup)